Ćwiczenie: od komentarza do działania

Przygotowanie, krok 0 i 8 etapów | praca indywidualna | Python i Flask | VS Code i Copilot CLI

Warsztat służy samodzielnej nauce. **Zawsze sprawdzaj, co AI robi w twoim projekcie — nie poprzestawaj na jego zapewnieniu, że skończyło.** Czytaj proponowane zmiany, uruchamiaj testy i sam wypróbuj aplikację. Wyniki i pliki tworzone przez skrypty służą tobie: pomagają zauważyć błędy, porównać zachowanie przed zmianą i po niej oraz zdecydować, co poprawić.

Ten dokument czytasz samodzielnie, nie dołączasz go do kontekstu agenta. Przed pierwszym zleceniem dodaj na początku własnych `.github\copilot-instructions.md` stałą regułę: agent ma ignorować `lab.md`, `lab.pdf`, `lab-speaker.md` i `lab-speaker.pdf` — bez odczytu, przeszukiwania, edycji ani przekazywania ich subagentom, także przez linki z zadań. Ma korzystać z przekazanego polecenia, wskazanego pliku w `issues`, `starter\brief.md`, `starter\quality-standards.md`, kodu i testów. Nie ma aktualizować materiałów warsztatu ani PDF-ów po zmianie aplikacji. Jeśli brakuje szczegółu kroku, ma poprosić cię o konkretny fragment. Przekazuj wybrany prompt, nie cały dokument; usuń materiały warsztatu z automatycznych załączników edytora. To zasada pracy, nie techniczna blokada dostępu. Nie usuwaj plików ani nie dodawaj ich do `.gitignore`.

Mapa kroku 0 i ośmiu etapów

Oznaczenia orientacyjnej długości pracy: `*` — krócej, `**` — dłużej. To porównanie zakresu pracy, nie limit czasu.

Najpierw przygotuj środowisko — `*` (jeśli narzędzia i dostęp do usług są już przygotowane). Potem wykonaj **krok 0 — diagram projektu** — `*`: w czacie Copilota w Visual Studio Code poproś jednym poleceniem o diagram Mermaid obecnego projektu. To wprowadzenie przed C1, bez zmiany numeracji etapów i modułów. W każdym etapie: **ustal oczekiwanie → sprawdź stan → przejrzyj zmianę AI → sam sprawdź wynik**.

1. **Instrukcje (własne zasady pracy). * Start:** skopiuj paczkę do własnego folderu wraz z `issues`, otwórz tylko tę kopię i zapisz powyższą regułę wyłączenia lab z kontekstu. **C1:** uzupełnij własne zasady i obejrzyj punkt odniesienia — `#iss001`. Testy uruchom przez `scripts\test.ps1`, a raport początkowy utwórz osobno przez `scripts\evaluate.ps1` z parametrem `-Output`. **Sprawdź:** po dodaniu instrukcji wykonaj cztery próby w osobnych sesjach: plan zmiany usługi, plan testu API, korektę Markdown i plan dla `#iss002`. Potwierdź zakres reguł oraz odczyt zadania; porównaj jedną sugestię agenta z wymaganiami.
2. **Naprawa błędu (prywatność i kontekst). * C2:** w trybie Plan zaplanuj test `other + actionable`, przełącz tryb tylko na lokalny zapis planu, wróć do Plan i zweryfikuj przekazanie kontekstu w nowej sesji. Potem osobno popraw maskowanie przez własną edycję w VS Code z podpowiedziami Copilota, bez agenta — `#iss002`; `actionability` wdrożysz dopiero w etapie 4. Po próbach edycji znajdziesz pomocnicze przykłady całego pliku maskowania i testu telefonu z myślnikami do porównania z własną zmianą. **Sprawdź:** tryby CLI, plik `plans\c2-actionability.md`, źródła, dokładne asercje i rozdzielenie faktów od wymagań; po naprawie maskowania test czerwony → zielony; dane osobowe nie trafiają do dostawcy ani szkicu.
3. **Skille i refaktoryzacja (powtarzalne testy i pomiar). ** C3:** przeczytaj i skopiuj gotowe wzorce skilli albo utwórz własne, następnie uruchom i sprawdź obie procedury; zastosuj `K1`: oddziel reguły `demo` od integracji `live` bez zmiany zachowania — `#iss003`. **Sprawdź:** drugą wersję promptu, działanie skilli i niezmienione wyniki 25 przypadków przed/po refaktoryzacji.
4. **Potrzeba działania (szkic tylko dla problemu). ** C4 i C5:** przećwicz cel (`/goal`) i tryb `/autopilot`, porównaj warianty `/fleet`, sprawdź plan i dodaj `actionability` — `#iss004`. Plan to opis zmian reguł rozpoznawania potrzeby działania, nie projekt UI. Powstaje w rozmowie na podstawie planu C2 i kodu po K1; przed wykonaniem przekaż go do krytyki C5. Wybrany wariant UI zachowaj na etap 5. **Sprawdź:** rzeczywiste delegacje, decyzję po krytyce oraz nowe testy pochwały, problemu, niejasnej sprawy i błędu dostawcy.
5. **Interfejs (czytelna decyzja i ponowienie). **** Oddziel HTTP od widoku, pokaż wynik po polsku — `#iss005`. **Sprawdź:** czy rozumiesz wynik bez otwierania JSON, możesz ponowić żądanie i rozpoznać poprzedni wynik; sprawdź też obsługę klawiaturą oraz widok o szerokości 375 px.
6. **CLI i pomiar (walidacja CSV i jakość). ** C6:** wykonaj skill CSV, sprawdź krok po kroku komentarz mieszany i pomiar — `#iss006`. CSV to osobne ćwiczenie: Copilot prowadzi procedurę z zasobami skilla, a lokalny skrypt sprawdza dane, z którymi porównasz odpowiedź agenta. **Opcjonalny krok:** po próbie w aplikacji przełącz ją z `demo` na rzeczywisty LLM przez Microsoft Entra; zachowaj osobne wyniki `live`. **Sprawdź:** działanie skilla CSV bez zmiany danych; macro-F1 i trafność priorytetów, obie miary >=0,90, w tym samym trybie aplikacji i dla `live` na tym samym modelu klasyfikującym; zbiór kontrolny osobno.
7. **Przegląd (uwagi poparte testem). * Powrót do C5:** przejrzyj zmiany z etapów 1–6; porównaj je z `lab-start`, jeśli go utworzyłeś — `#iss007`. **Sprawdź:** testy, test kontrolny po restarcie i kilka istotnych miejsc w kodzie.
8. **Plugin (sprawdzenia jednym zleceniem). *** Spakuj skille i role, uruchom koordynatora — `#iss008`. **Sprawdź:** dwie delegacje, uruchomione skille i podsumowanie.

Moduły C1–C6 są numerowane według pierwszego wykonania; lokalny przegląd w etapie 7 wraca do C5. Spec Kit i końcowe TUI są opcjonalne — dopiero po rdzeniu. Najpierw zainstaluj zatwierdzone narzędzia Spec Kit i Brownfield,

odtwórz specyfikacje z kodu i ustaw właściwą jako aktywną (`#iss010`). Rozpoznaj całą nieznaną aplikację: wykonaj skan, utrwal mapę rozwiązania i specyfikacje odkrytych funkcji. To symulacja przejęcia kodu bez wcześniejszej wiedzy o tym, co robi. Następnie opisz i dodaj konsolowe wpisywanie komentarza z oceną w JSON jak na WWW (`#iss009`, AC-11). To nowa funkcja, nie monitor ewaluacji.

Osobny pokaz prowadzącego dotyczący GitHub Agentic Workflows i Azure nie należy do ćwiczenia ani do paczki uczestnika. Uczestnik nie konfiguruje jego przepływów pracy, usług ani uprawnień; poniższe ćwiczenie nadal wykonuje lokalnie.

Brief opisuje wymagania, a standardy jakości — co sprawdzić w aplikacji. Oznaczenia AC pomagają znaleźć konkretne oczekiwane zachowanie. PASS, FAIL i BLOCKED opisują wyniki narzędzi: czy sprawdzenie się powiodło, wykryło problem, czy nie mogło się wykonać. Nie są oceną uczestnika. Oczekiwany FAIL może potwierdzać, że test prawidłowo wykrywa znany błąd — zawsze sprawdzaj, czego dotyczy wynik.

Jak czytać oznaczenia i polecenia

Etap porządkuje pracę nad aplikacją, a **moduł C1–C6** oznacza ćwiczoną technikę pracy z Copilotem. **AC** to kryterium akceptacji, czyli konkretne oczekiwane zachowanie. **K1/K2** opisują podział odpowiedzialności kodu, a **U1–U3** — problemy użyteczności. Ich treść znajdziesz w standardach jakości; oznaczenia nie są komendami.

W tekście **prompt** oznacza zlecenie dla AI, **skill** — procedurę z instrukcją i ewentualnymi zasobami, **subagent** — osobnego agenta wykonującego wydzielone zadanie, a **plugin** — pakiet takich rozszerzeń. **API** to interfejs komunikacji z aplikacją, **UI** — jej widok dla użytkownika, **LLM** — model językowy, a **PII** — dane pozwalające zidentyfikować osobę. W tym warsztacie maskowanie obejmuje tylko formaty wymienione w briefie.

Bloki `powershell` uruchamiaj w terminalu, a prompty z bloków `text` wklejaj do wskazanego czatu. Polecenia zaczynające się od `/`, np. `/review`, należą do interfejsu Copilota, nie do PowerShell. Bloki `python`, `json` i `ini` przedstawiają kod lub zawartość pliku; wykonuj z nimi czynność opisaną nad blokiem, zamiast wklejać je do terminala. Tekst w nawiasach, np. `[Tutaj wklej notatkę.]`, zastąp własną treścią. Nazwy menu i skróty mogą zależeć od wersji narzędzia oraz ustawień klawiatury.

Lokalne zadania zamiast Azure DevOps / Jira

Pliki `issues\iss001.md`–`issues\iss010.md` symulują elementy pracy w Azure DevOps / Jira: mają typ, stan początkowy `New`, opis, zadanie i wskazówki do sprawdzenia wyniku. Nie łączymy się z Azure DevOps ani Jira i nie tworzymy prawdziwych zgłoszeń. Zadania zapisujemy wyłącznie w Markdown — nie generujemy dla nich PDF-ów. Link w PDF ćwiczenia prowadzi do pliku `.md`, nie do innego PDF-a.

W etapie 1 dodaj do własnych `.github\copilot-instructions.md` zasadę: gdy wiadomość zawiera `#iss<id>`, agent przed analizą lub zmianą odczytuje `issues\iss<id>.md`. Zachowuje cyfry ID, również zera początkowe; brak pliku zgłasza jawnie. To konwencja instrukcji, nie funkcja integracji. Przed utworzeniem zasad podaj agentowi ścieżkę pliku wprost.

Przykład po włączeniu instrukcji:

```
Przeczytaj #iss002 i zaplanuj testy. Bez edycji i uruchamiania poleceń.
```

Sprawdź rzeczywisty odczyt `issues\iss002.md` w narzędziach lub References, nie tylko odpowiedź modelu. `New` opisuje stan początkowy; wykonanie zadania sprawdzaj testami i zachowaniem aplikacji.

Cel

Celem jest nauczenie się **dostosowywania GitHub Copilota do projektu**:

- zapisywania stałych zasad jako instrukcji;
- ograniczania instrukcji do właściwych plików;
- budowania skilli z deterministycznymi skryptami;
- świadomego zarządzania kontekstem i kontynuowania pracy w nowej sesji;
- używania celu, subagentów i drugiej opinii;
- zebrania powtarzalnych sprawdzeń w lokalnym pluginie.

Aplikacja Flask jest materiałem ćwiczebnym. Nie budujesz systemu od zera i nie tworzysz prawdziwych zgłoszeń. Pracujesz lokalnie, na danych syntetycznych, bez wysyłania zmian do repozytorium zdalnego, tworzenia pull requestów, GitHub Actions i agenta chmurowego Copilota.

W każdym etapie korzystaj z kryteriów, żeby wiedzieć, na co zwrócić uwagę:

kryterium → test lub próba użytkownika → zmiana → sprawdzenie wyniku

Nie polegaj wyłącznie na deklaracji agenta, kodzie wyjścia procesu Copilot ani statusie HTTP 200: obejrzyj konkretne wartości i zachowanie aplikacji.

Jak wrócić do pracy, gdy zgubisz wątek

Znajdź swój etap w mapie i otwórz wskazane lokalne zadanie. Sprawdź jego **warunek startu**: numer etapu nie dowodzi, że poprzedni krok się udał. W etapach poniżej rozróżniamy zlecenie dla agenta, oczekiwany wynik i sprawdzenie wykonywane przez ciebie.

1. Ustal, czy teraz **planujesz**, **zmieniasz**, czy **sprawdzasz**. Dobry plan nie oznacza zgody na implementację. W próbie „bez poleceń powłoki" agent może czytać wskazane źródła, ale nie uruchamia testów ani powłoki.
2. Zobacz, co naprawdę powstało: odpowiedź w rozmowie, zmiana pliku, wykonanie narzędzia czy raport z bieżącego przebiegu. Każde z nich mówi coś innego o tym, co agent rzeczywiście zrobił. Istniejący test to nie wykonany test, a planowana asercja to nie wynik.
3. Porównaj wynik z kryterium danego kroku. Jeśli nie pasuje, wróć do najmniejszej nieudanej próby, zamiast zlecać „napraw wszystko". Nie obniżaj wymagań i nie kasuj dotychczasowej pracy.
4. Gdy brakuje danych, agent powinien nazwać brak i wskazać, jak go wyjaśnić. Brak narzędzia, dostępu lub wykonania oznacza część niepotwierdzoną, nie sukces. Przy prośbie o pomoc podaj etap, oczekiwanie, rzeczywisty wynik i bezpieczny fragment błędu.

Możesz wkleić poniższy prompt, uzupełniając nawiasy własnymi informacjami:

```
Pracuję nad [cel i lokalne zadanie]. Ostatni potwierdzony wynik:
[wynik oraz skąd go znam]. Teraz oczekuję [konkretne zachowanie],
ale otrzymuję [wynik lub błąd]. Na podstawie wskazanych źródeł
oddziel fakty od przypuszczeń. Wskaż jeden następny krok
i sposób jego sprawdzenia. Na razie bez edycji i poleceń powłoki.
```

Przygotowanie środowiska

Wymagane są Git, Python 3.11+, PowerShell, VS Code z rozszerzeniami Python, Python Debugger i Copilot oraz osobno zainstalowany i uwierzytelniony Copilot CLI.

Własna kopia, potem dalsze kroki

Jeśli masz repozytorium źródłowe warsztatu, przed uruchomieniem agenta skopiuj niezbędne pliki skryptem do **nowego folderu poza tym repozytorium**:

```
.\scripts\export-participant.ps1 -Destination ..\triage-participant
Set-Location ..\triage-participant
code .
```

Jeżeli dostałeś już przygotowaną paczkę, skopiuj lub rozpakuj ją do własnego folderu i otwórz ten folder w VS Code — nie uruchamiaj eksportu ponownie. Dalej pracuj wyłącznie w tej kopii, również w terminalu Copilot CLI. Punktem wejścia jest `lab.md`; folder `issues` z zadaniami Markdown pozostaje obok niego. To symulacja Azure DevOps / Jira, nie integracja. Wzorce w `starter\copilot` pozostają nieaktywne; C1 wykonasz samodzielnie. Skrypt nie nadpisuje istniejącego folderu, nie kopiuje `.git`, `.env`, środowiska ani raportów i niczego nie instaluje.

Po wklejeniu polecenia z PDF sprawdź je przed uruchomieniem: spacje przed opcjami, cudzysłowy i podział na linie mają znaczenie. W razie wątpliwości porównaj je z blokiem w `lab.md`. Ilustracje pokazują przykładowe sesje autora: nie kopiuj z nich ścieżek, nazw modeli ani komunikatów jako wyników własnej pracy.

Opcjonalny punkt startowy Git

Lokalny commit `lab-start` ułatwia późniejszy przegląd zmian. Jeśli taki punkt odniesienia ci się przyda, utwórz go przed C1. Poniższe polecenia zakładają kopię bez istniejącego repozytorium Git:

```
git init -b workshop
git add .
git status --short
git --no-pager diff --cached --stat
```

Sprawdź listę plików; nie dodawaj sekretów ani raportów. Po sprawdzeniu:

```
git commit -m "Zachowujemy projekt startowy, aby porownywac kolejne zmiany."
git tag lab-start
git rev-parse lab-start
```

Jeśli Git zgłasza brak tożsamości autora, skonfiguruj własną zgodnie z zasadami organizacji; nie używaj fikcyjnej. Nie przesuwaj tagu `lab-start`, jeśli go utworzysz.

Przeglądaj zmiany lokalnie i twórz commity wtedy, gdy ułatwiają ci porównanie wersji lub powrót do wcześniejszego stanu. Nie wysyłamy zmian do repozytorium zdalnego.

Gdy zmiana się nie uda, zatrzymaj edycje agenta i zachowaj bieżącą kopię. Możesz rozpocząć ponowną próbę w **nowym** folderze z wybranego commita, bez resetowania ani kasowania nieudanej pracy:

```
git log --oneline
# Zastap <SHA> identyfikatorem wybranego commita.
git worktree add -b retry-stage-4 ..\triage-retry-stage-4 <SHA>
```

Przed dalszą pracą otwórz nowy folder i przygotuj w nim środowisko poniższymi poleceniami. `.venv`, `.env` i ignorowane raporty nie są przenoszone przez Git.

Uruchomienie projektu

W katalogu projektu uruchom:

```
python --version
copilot --version
.\scripts\setup.ps1
.\scripts\test.ps1
```

W VS Code wybierz interpreter `.venv\Scripts\python.exe`. `.venv` to lokalne środowisko Pythona z zależnościami tego projektu; wybór innego interpretera może powodować błędy importu mimo udanej instalacji. Nie zmieniaj skonfigurowanego rejestru pakietów i nie obchodź polityki uruchamiania skryptów.

Stan początkowy: dokładnie `70 PASS / 1 FAIL`. Kod wyjścia testów `1` jest oczekiwany. Przy innym wyniku lub błędzie importu sprawdź interpreter i przygotowanie środowiska; jeśli utkniesz, poproś prowadzącego o pomoc.

Uruchom aplikację:

```
.\scripts\run.ps1
```

Otwórz `http://127.0.0.1:5000`, a w drugim terminalu wykonaj:

```
.\scripts\smoke.ps1
.\scripts\evaluate.ps1 -Mode demo -Output reports\baseline-demo.json
```

Tryb `demo` jest deterministyczny i nie używa modelu językowego. Copilot nadal wymaga dostępu do zatwierdzonej usługi AI.

Rozdziel trzy miejsca pracy. W edytorze zmieniasz pliki, w czacie wydajesz zlecenia Copilotowi, a w terminalu uruchamiasz skrypty. Terminal z serwerem pozostaw otwarty; testy i pomiary uruchamiaj w drugim. Serwer zatrzymasz przez `Ctrl+C` w jego terminalu. Po zmianie aplikacji uruchom go ponownie — skrypt nie włącza automatycznego przeładowania kodu. Przed przejściem między agentem w VS Code a CLI zakończ poprzednie edycje: obie sesje pracują na tych samych plikach.

Opcjonalny tryb `live`

Konfiguruj go tylko z parametrami przekazanymi przez organizatora. Aplikacja nie przełącza się automatycznie między `live` i `demo`. Pełną procedurę przygotowania zależności, konfiguracji, logowania przez Microsoft Entra i pierwszego wywołania LLM znajdziesz w kroku live etapu 6. Możesz wykonać ją już teraz, jeżeli cały warsztat zaczynasz w `live`.

W `.env` wybierz jedną zatwierdzoną metodę:

- `OPENAI_AUTH_MODE=api-key` i `OPENAI_API_KEY`;
- `OPENAI_AUTH_MODE=azure-cli`, lokalne `az login` oraz właściwy `AZURE_TOKEN_SCOPE`.

Użyj dokładnego URL i nazwy wdrożenia od organizatora. Dla Azure OpenAI API v1 URL zwykle kończy się `/openai/v1/`. `OPENAI_TEMPERATURE=default` pomija parametr temperatury, jeśli wdrożenie go nie obsługuje.

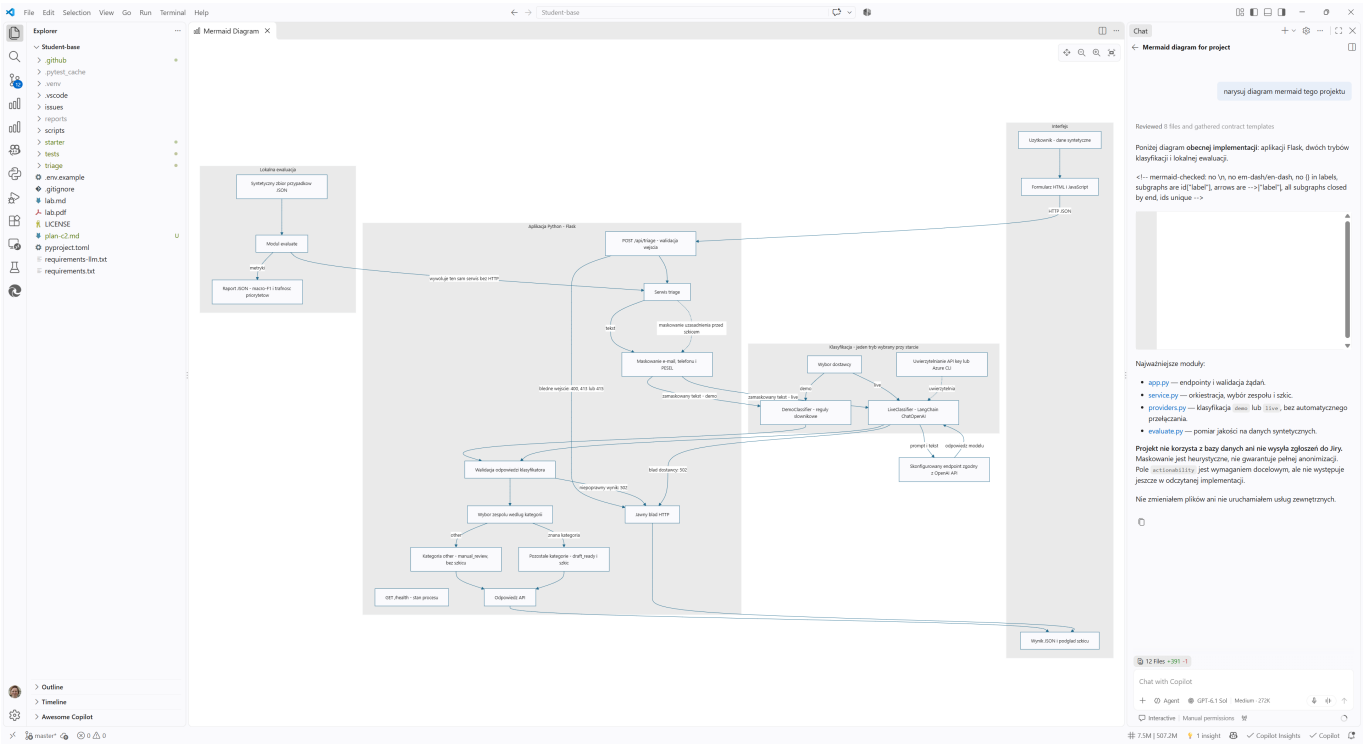
Wyników `demo` i `live` nie porównuj jako jednego eksperymentu.

Krok 0 — diagram projektu w Visual Studio Code

Po przygotowaniu środowiska, we własnej kopii projektu i z zapisaną regułą wyłączenia lab z kontekstu, otwórz czat Copilota w Visual Studio Code. Przed C1 wyślij jedno polecenie:

`Narysuj diagram Mermaid tego projektu.`

Powinien powstać diagram podobny do poniższego przykładu. Układ i nazwy elementów mogą się różnić; chodzi o obraz obecnej implementacji, nie projekt przyszłych zmian. Mermaid to tekstowy zapis diagramu, który edytor może wyświetlić graficznie. Obejrzyj podgląd i odszukaj interfejs, aplikację Flask, klasyfikację oraz lokalną ewaluację. Porównaj przynajmniej jedno połączenie z kodem: przekonujący rysunek może zawierać relację, której aplikacja nie realizuje.



Rysunek 1: Przykładowy diagram Mermaid projektu w Visual Studio Code po wysłaniu jednego polecenia.

Etap 1 — instrukcje i punkt odniesienia

Opis zadania i kryteria: `#iss001`.

C1. Instrukcje repozytorium i reguły dla plików

Ustal trzy zasady: wspólną dla repozytorium, dotyczącą aplikacji i dotyczącą testów. Umieść je w odpowiednich instrukcjach, żeby agent mógł z nich korzystać przy kolejnych zmianach.

Utwórz:

- `.github\copilot-instructions.md` — krótkie zasady wspólne;
- `.github\instructions\triage-python.instructions.md` z `applyTo: "triage/**/*.py"`;
- `.github\instructions\triage-tests.instructions.md` z `applyTo: "tests/**/*.py"`;
- `triage\templates\AGENTS.md` — wyłącznie lokalne zasady UI.

Nie kopiuj całego briefu do instrukcji i nie twórz głównego `AGENTS.md` powielającego zasady repozytorium. Istniejące zasady firmowe scalaj, nie nadpisuj. Zachowaj na początku `.github\copilot-instructions.md` regułę wyłączenia czterech plików lab, dodaną przed pierwszym zleceniem. Pozostałe zasady dopisz pod nią; nie zastępuj nią własnej pracy nad C1. Stałe zasady ochrony danych umieść w instrukcjach Copilota, zamiast powtarzać je w każdym prompcie. W promptach określaj zakres bieżącej pracy, np. sam plan bez edycji i uruchamiania poleceń.

```
Na podstawie starter\brief.md, starter\quality-standards.md,
scripts\test.ps1 i scripts\evaluate.ps1 utwórz krótkie instrukcje.
Zasady wspólne mają wymagać pracy lokalnej, danych syntetycznych,
.venv, PowerShell i uruchamiania testów. Mają zabraniać osłabiania
testów oraz tworzenia commitów bez mojej zgody.
Dodaj zasadę automatycznego odczytu issues\iss<id>.md
dla odwołań #iss<id>: tylko cyfry, bez zmiany zer początkowych,
przed analizą lub edycją; brak pliku zgłaszaj jawnie,
bez Azure DevOps i GitHub.
Zasady triage/**/*.py mają dotyczyć typów, walidacji danych
na granicach komponentów, obsługi konkretnych wyjątków
i rozdzielenia logiki domenowej od Flask oraz dostawcy.
Zasady tests/**/*.py mają wymagać asercji powiązanych z AC,
danych syntetycznych i braku skip, xfail oraz live.
triage\templates\AGENTS.md ma wymagać zachowania polskich
komunikatów, obsługi klawiatury, aria-live, textContent,
stanu poprzedniego wyniku i widoku 375 px.
Nie zmieniaj aplikacji.
```

Cztery próby po dodaniu własnych instrukcji Sprawdzasz **własne zapisane instrukcje**, nie zdolność agenta do powtórzenia reguł z promptu. Wykonaj poniższe próby dopiero po utworzeniu plików C1. Nie aktywuj jeszcze wzorców z biblioteki. W każdej próbie sprawdź również, czy agent nie odczytuje żadnego z czterech wyłączonych plików lab. W próbie `#iss002` link do instrukcji ćwiczenia nie jest zgodą na jej otwarcie.

Przed każdą próbą rozpocznij **osobną, pustą rozmowę** w tej samej kopii projektu. W CLI użyj `/clear`, a w VS Code otwórz nową rozmowę. Sama zmiana modelu ani skrócenie historii rozmowy nie zerują kontekstu. Nie wklejaj odpowiedzi z poprzedniej próby ani treści instrukcji do promptu; usuń niepotrzebne załączniki. W VS Code nie dołączaj automatycznie otwartego pliku Pythona do próby Markdown.

W każdej próbie wolno czytać potrzebne pliki, ale **nie wolno zapisywać zmian, uruchamiać testów ani innych poleceń powłoki**. Korekta Markdown także ma pozostać propozycją w odpowiedzi. Nie wybieraj wykonania planu. Jeśli agent prosi o zgodę na edycję lub uruchomienie, zatrzymaj go. Ogólna zasada uruchamiania testów po zmianie nie oznacza ich uruchamiania w próbie obejmującej wyłącznie plan.

Sprawdź zarówno użyty kontekst, jak i samą odpowiedź:

- **Kontekst:** w VS Code rozwiń References / użyty kontekst i wywołania narzędzi; w CLI sprawdź `/instructions` oraz historię odczytów. Lista dostępnych lub włączonych plików nie dowodzi jeszcze, że reguła pasowała do konkretnego zadania.
- **Odpowiedź:** znajdź konkretny fragment planu zgodny z właściwą regułą. Samo „zastosowałem instrukcje” nie wystarcza. Jeśli interfejs nie pokazuje użytego kontekstu, nie zakładaj, że agent go wykorzystał.

Próba 1 — plan zmiany w usłudze W nowej rozmowie wyślij:

```
Przeczytaj triage\service.py. Zaplanuj małą refaktoryzację:
wydzielenie budowania szkicu bez zmiany obecnego zachowania.
```

```
Wskaż granice zmiany, zachowywany kontrakt, ryzyka oraz testy
do późniejszego uruchomienia. Oddziel fakty od propozycji.
Na końcu wskaż instrukcje, które wpłynęły na plan, i podaj
po jednym przykładzie ich zastosowania.
Tylko odczyt i odpowiedź: bez edycji i poleceń powłoki.
```

Jak sprawdzić: potwierdź odczyt `triage\service.py`. Do tego zadania pasują zasady wspólne oraz `triage-python.instructions.md` z `applyTo: "triage/**/*.py"`. Szukaj w planie zachowania typów, walidacji odpowiedzi dostawcy oraz rozdzielenia logiki domenowej od Flask i kodu integracji z dostawcą. Agent powinien odwoływać się do istniejącego kodu, nie wymyślać brakującej walidacji ani proponować nowych warstw bez potrzeby.

Oczekiwany wynik: plan zachowuje maskowanie przed wywołaniem klasyfikatora, obecne pola odpowiedzi i warunki tworzenia szkicu. Nie wprowadza jeszcze `actionability` ani nie naprawia błędu etapu 2. Wymienia testy jako planowane, nie zakończone sukcesem. Jeśli agent odczyta także testy, reguły testowe mogą być zasadne dla tej części pracy; nie traktuj samego ich pojawienia się jako błędu zakresu.

Próba 2 — plan testu API W kolejnej nowej rozmowie wyślij:

```
Przeczytaj tests\test_api.py i AC-05 w
starter\quality-standards.md. Zaplanuj test awarii dostawcy
dla POST /api/triage. Podaj przygotowanie, żądanie i konkretne
asercje. Najpierw sprawdź, co już pokrywają istniejące testy:
jeśli wystarczają, uzasadnij brak potrzeby dodawania duplikatu.
Wskaż zastosowane instrukcje i ich wpływ na plan.
Tylko odczyt i odpowiedź: bez edycji i poleceń powłoki.
```

Jak sprawdzić: potwierdź odczyt testów i AC-05. Do pliku `tests\test_api.py` pasują zasady wspólne i `triage-tests.instructions.md` z `applyTo: "tests/**/*.py"`. Plan ma używać syntetycznych danych wejściowych, atrapy dostawcy zamiast połączenia `live` i asercji powiązanych z kryterium. Nie powinien proponować `skip`, `xfail` ani zmiany oczekiwań na sukces.

Oczekiwany wynik: odpowiedź uwzględnia HTTP `502`, jawny `error`, brak szkicu i brak zastąpienia błędu wynikiem `demo`; samo sprawdzenie kodu HTTP nie wystarcza. Rozróżnia awarię dostawcy od niepoprawnej kategorii odpowiedzi i zauważa istniejące pokrycie obu sytuacji. Jeśli istniejący test wystarcza, sprawdź wskazane przez agenta asercje zamiast dodawać drugi test tego samego zachowania.

Próba 3 — korekta akapitu Markdown W kolejnej nowej rozmowie wyślij:

```
Przeczytaj starter\brief.md. Zaproponuj jaśniejszą redakcję
punktu zaczynającego się od „draft_ready" w części
„Sentyment, przegląd i szkic". Zachowaj znaczenie i identyfikator.
Pokaż tylko proponowany tekst oraz krótkie uzasadnienie.
Dodaj, które instrukcje mają tu zastosowanie, a które nie.
Nie analizuj kodu aplikacji ani testów.
Tylko odczyt i odpowiedź: bez edycji i poleceń powłoki.
```

Jak sprawdzić: kontekstem ma być wskazany Markdown i zasady wspólne, a nie pliki aplikacji dołączone z poprzedniej rozmowy. Reguły `triage/**/*.py` i `tests/**/*.py` nie pasują do tego dokumentu; lokalne zasady `triage\templates\AGENTS.md` również go nie obejmują. Jeśli masz osobne instrukcje Markdown lub zasady firmowe, mogą nadal obowiązywać — nie wyłączaj ich na potrzeby próby.

Oczekiwany wynik: propozycja nadal jasno mówi, że `draft_ready` nie oznacza zatwierdzenia ani wysłania zgłoszenia. Zachowuje zapis identyfikatora jako kodu Markdown. Nie dodaje wymagań typowania Pythona, fixture'ów ani uruchamiania pytest do korekty tekstu. Nie uznawaj wzmianki o testach wynikającej z zasad wspólnych za dowód błędnego zakresu; problemem jest narzucenie reguł plików Pythona niezwiązanemu zadaniu Markdown.

Próba 4 — plan testów dla lokalnego zadania W kolejnej nowej rozmowie wskaż zadanie **wyłącznie przez jego ID**, bez ścieżki i bez dołączania pliku. Użyj poniższego promptu, aby sprawdzić regułę utworzoną w C1:

```
Przeczytaj #iss002 i zaplanuj testy. Na początku potwierdź,
jaki plik zadania odczytałeś, oraz streść jego cel i kryteria.
Uwzględnij wymagania powiązane z zadaniem. Dla każdego przypadku
podaj dane syntetyczne, co obserwować i oczekiwany wynik.
Rozdziel stan obecny, wymagania i planowane sprawdzenia.
Nie naprawiaj błędu. Bez edycji i poleceń powłoki.
```

Jak potwierdzić odczyt: rozwiń wywołanie narzędzia i sprawdź, czy otwarto dokładnie `issues\iss002.md`, z zerami w ID, **przed analizą i przygotowaniem planu**. W VS Code sprawdź również References. Sam wynik wyszukiwania

nazwy pliku albo zdanie „przeczytałem zadanie” nie wystarczają. Odpowiedź powinna zgadzać się z opisem i kryteriami rzeczywiście odczytanego pliku. Wywołanie GitHub lub Azure DevOps, odczyt `iss2.md` albo zgadywanie treści to sygnał, żeby poprawić instrukcję i ponowić próbę.

Oczekiwany wynik: plan uwzględnia AC-01 oraz formaty z briefu: `500600700`, `+48500600700`, `500 600 700`, `+48 500 600 700`, `500-600-700`, prosty e-mail i syntetyczny ciąg 11 cyfr. Zawiera połączenie e-maila i telefonu oraz przypadki ochrony zwykłych ilości i identyfikatora sklepu. Dla PII wskazuje dwa miejsca obserwacji: argument przekazany dostawcy i powstały szkic, nie tylko widok aplikacji. Zachowuje treść biznesową oraz pierwotne oczekiwanie czerwonego testu.

Plan ma rozróżniać oczekiwany czerwony wynik przed naprawą od zielonego po naprawie i późniejszych regresji; żaden z nich nie został wykonany w tej próbie. Powinien też zauważyć warunek startu zadania — punkt odniesienia z etapu 1 — oraz późniejszą notatkę do 250 słów i jej weryfikację w nowej sesji. Próba C1 nie wykonuje zadania 002; naprawa, regresje i notatka należą do etapu 2.

Co zrobić, gdy próba nie potwierdza stosowania instrukcji Sprawdź zapisanie plików, katalog otwartego projektu, nazwy instrukcji i wzorce `applyTo`. Dla odwołania do zadania sprawdź regułę w `.github\copilot-instructions.md` oraz obecność `issues\iss002.md`. Gdy pliku brakuje, agent ma jawnie zgłosić brak, nie odtwarzać jego treści. Popraw własną konfigurację i powtórz tylko daną próbę w nowej rozmowie, z tym samym promptem. Nie dopisuj do niego brakującej reguły, bo sprawdzałbyś wtedy prompt, a nie działanie zapisanych instrukcji.

W VS Code Local zagnieżdżone `AGENTS.md` może wymagać ustawienia obszaru roboczego `chat.useNestedAgentsMdFiles`; nie zmieniaj ustawień globalnych. Cztery powyższe próby nie potwierdzają zastosowania zasad UI: ich lokalny zakres obejrzyj osobno przy pracy z szablonem.

Porównaj własny wynik dopiero po próbie z nieaktywnymi wzorcami w bibliotece.

Na co zwrócić uwagę Upewnij się, że instrukcje obejmują trzy obszary: wspólne zasady, kod aplikacji i testy. W świeżych sesjach sprawdź, czy właściwe zasady pasują do pracy z kodem i testami, a niepotrzebne nie są stosowane do zadania Markdown. Porównaj jedną sugestię agenta z wymaganiami i popraw ewentualne rozbieżności. W tych próbach akceptacja propozycji nie jest zgodą na edycję aplikacji. Przyjrzyj się także lokalnym zasadom UI.

Zadanie aplikacyjne: punkt odniesienia

Testy i pomiar to dwa osobne kroki. Poprawna ścieżka do testów to `scripts\test.ps1`, nie `script\tests.ps1`. Ten skrypt uruchamia pytest i **nie tworzy** `reports\baseline-demo.json`. Raport metryk zapisuje dopiero `scripts\evaluate.ps1` z parametrem `-Output`; bez tego parametru wynik pomiaru jest tylko wypisywany w terminalu.

Wykonaj poniższe kroki w PowerShell, w katalogu głównym swojej kopii projektu, po przygotowaniu `.venv`.

1. Uruchom testy i od razu odczytaj ich kod wyjścia:

   ```
   .\scripts\test.ps1
   $LASTEXITCODE
   ```

   W niezmienionym projekcie oczekuj `70 PASS / 1 FAIL` i kodu `1`. To celowy błąd prywatności, nie powód pomijania pomiaru. Nie łącz testów i ewaluacji przez `&&`, bo oczekiwany kod `1` zatrzymałby taką sekwencję. Inny błąd wyjaśnij przed dalszą pracą.

2. Sprawdź, czy raport powstał już podczas przygotowania środowiska:

   ```
   Test-Path .\reports\baseline-demo.json
   ```

   Jeśli otrzymasz `False`, uruchom osobno pomiar początkowy:

   ```
   .\scripts\evaluate.ps1 -Mode demo `
       -Output reports\baseline-demo.json
   $LASTEXITCODE
   ```

   Po udanym pomiarze oczekuj kodu `0` i zapisanego pliku. Skrypt sam tworzy katalog `reports`, jeśli go brakuje. Pomiar nie wymaga uruchomionego serwera Flask — wywołuje klasyfikator lokalnie. Kod `0` tego polecenia oznacza ukończenie pomiaru, nie osiągnięcie progów jakości z etapu 6.

   Jeśli raport już istnieje, sprawdź, czy pochodzi z pomiaru początkowego tej kopii projektu. Nie nadpisuj go po zmianach aplikacji. Sama obecność pliku nie dowodzi jego aktualności ani pochodzenia.

3. Otwórz raport i obejrzyj najważniejsze pola:

   ```
   Get-Item .\reports\baseline-demo.json
   Get-Content .\reports\baseline-demo.json -Raw |
   ```

```
ConvertFrom-Json |
Select-Object mode, cases, macro_f1, priority_accuracy
```

Dla powyższego polecenia oczekuj `mode=demo` i `cases=25`. Odczytaj rzeczywiste wartości `macro_f1` i `priority_accuracy`; nie zakładaj z góry osiągnięcia progów. `macro_f1` to średnia miara jakości rozpoznawania pięciu kategorii, a `priority_accuracy` to odsetek poprawnych priorytetów. Nie są tym samym — wyjaśnienie i przykład znajdziesz przy pomiarze końcowym. Jeśli pliku nie ma, sprawdź komunikat i kod wyjścia ewaluacji oraz katalog roboczy. Nie twórz pustego JSON ręcznie i nie uruchamiaj ponownie samych testów w oczekiwaniu na raport.

4. W uruchomionym formularzu sprawdź komentarz mieszany „Obsługa super, ale nie mogę zamówić towaru." oraz „Miła obsługa, dziękuję.". Jeśli serwer nie działa, uruchom `.\scripts\run.ps1 -Mode demo` w osobnym terminalu i otwórz `http://127.0.0.1:5000`.
5. Obejrzyj tryb aplikacji. Powyższy punkt odniesienia jest dla `demo`, bez modelu językowego. Jeśli osobno wykonujesz zatwierdzony wariant `live`, użyj innej nazwy raportu i zanotuj użyty model — raport metryk nie zawiera jego nazwy. Nie porównuj `live` z `baseline-demo.json`.
6. Spróbuj ustalić decyzję, priorytet, zespół i stan wysłania bez analizowania JSON. Zauważ, czego nie da się jeszcze łatwo odczytać z widoku — do tych problemów wrócisz w etapie 5.

Nie dodawaj jeszcze nowych czerwonych testów. Zachowaj `reports\baseline-demo.json`; w etapie 6 porównasz go z pomiarem końcowym w tym samym trybie. Pomiar końcowy zapisuj do innego pliku, aby nie nadpisać punktu odniesienia.

Etap 2 — diagnoza z kontrolowanym kontekstem

Opis błędu i kryteria: `#iss002`.

C2. Wybierz, sprawdź i przekaż kontekst

Cel: nauczyć się sprawdzać, skąd agent wie to, co pisze, oraz przekazać zadanie między sesjami bez przenoszenia niepotwierdzonych założeń. Zapiszesz **lokalny plan testów i sprawdzisz notatkę do nowej sesji**, żeby móc później wrócić do tych ustaleń. Na razie nie zmieniasz kodu ani nie uruchamiasz planowanych testów. Przećwiczysz też świadome przełączanie trybów Copilot CLI: planowanie → zapis pliku → ponowna analiza. Masz już własne instrukcje z C1 i punkt odniesienia. Nowe pole `actionability` wdrożysz dopiero w etapie 4; tutaj świadomie pytasz o wymaganie, którego projekt startowy jeszcze nie realizuje. Naprawa maskowania jest osobnym zadaniem poniżej.

Co oznacza `actionability` i dlaczego jest istotne `actionability` oznacza tutaj **potrzebę działania**: czy komentarz opisuje konkretny problem, którym należy się zająć, czy jest tylko informacją, czy też brakuje podstaw do decyzji. Rozdziel trzy pytania:

Pole	Pytanie	Przykład
`category`	Czego dotyczy sprawa?	`store_service` — obsługi sklepu
`sentiment`	Jaki jest wydźwięk komentarza?	`positive` — pozytywny
`actionability`	Czy są podstawy do działania?	`informational` — sama pochwała

Sama kategoria wskazuje obszar, ale nie dowodzi istnienia problemu. Sam sentyment opisuje ton, ale nie rozstrzyga, czy trzeba działać. „Miła obsługa, dziękuję” dotyczy sklepu, lecz nie wymaga szkicu zgłoszenia. „Produkt ma uszkodzone opakowanie” spokojnie opisuje konkretny problem. „Miła obsługa, ale składanie zamówień nie działa” zawiera pochwałę, jednak problem nadal wymaga działania.

W nowym wymaganiu z briefu obowiązują trzy wartości:

- `actionable` — konkretny problem w znanej kategorii: `draft_ready`, czyli szkic do oceny człowieka, nie wysłane zgłoszenie.
- `informational` — wyłącznie pochwała lub informacja w znanej kategorii: `no_action`, bez szkicu.
- `unclear` — brak podstaw do decyzji: `manual_review`, bez szkicu. Człowiek musi wyjaśnić sprawę; to nie to samo co „nic nie trzeba robić”.

To rozróżnienie ogranicza zbędne szkice dla pochwał, nie pozwala przeoczyć problemu ukrytego w miłym komentarzu i pozostawia niejasne sprawy człowiekowi. `review_status` opisuje wynikającą z tej oceny ścieżkę obsługi, a nie ton wypowiedzi ani potwierdzenie wykonania działania.

Dlaczego badamy `other + actionable`? Dostawca jednocześnie mówi „poza taksonomią / za mało informacji” i „konkretny problem do działania”. W tym warsztacie `other` ma pierwszeństwo: zachowujemy kategorię, normalizujemy `actionability` do `unclear`, kierujemy do ręcznej weryfikacji i nie tworzymy szkicu. To jawna, zachowawcza reguła briefu, nie uniwersalne znaczenie słowa `other`. Pozwala sprawdzić, czy aplikacja stosuje własne reguły, zamiast bezwarunkowo ufać ocenie dostawcy.

1. Wybierz źródła dla jednego pytania Rozpocznij nową rozmowę w swojej kopii projektu. Usuń przypadkowe załączniki i pozostałości poprzedniego zadania. Dołącz pliki lub wskaż ich dokładne ścieżki; dla standardów wybierz konkretne kryteria. Nie dołączaj całego repozytorium „na wszelki wypadek”.

Źródło	Co ma wyjaśnić
Brief, kontrakt początkowy i nowe wymaganie	Co jest dzisiaj, a co ma się zmienić
Kontrakty	Jakie pola przyjmuje i zwraca obecny kod
Usługa	Gdzie waliduje wynik, wybiera status i tworzy szkic
Testy API	Co już sprawdzają asercje i jak podstawiono dostawcę
Standardy, AC-04 i AC-05	Normalizacja sprzeczności oraz granica błędu kontraktu

AC-04 jest głównym kryterium; AC-05 chroni przed „naprawianiem” każdej błędnej odpowiedzi jako `other`. Pozostałe AC, UI i implementacja dostawcy nie są potrzebne do tego jednego planu. Aktywne instrukcje C1 pozostają zasadami pracy, nie dodatkowymi wymaganiami API.

W tej próbie nie wysyłaj `#iss002`: dotyczy maskowania i skierowałoby agenta do innego zadania. Jeśli wybrane źródła nie wystarczą, oczekuj wskazania konkretnego brakującego pliku i powodu jego odczytu, nie samodzielnego rozszerzenia zadania.

2. Zleć tylko plan Wykonaj tę próbę w Copilot CLI, w trybie Plan, żeby sprawdzić różnicę między planowaniem a zgodą na zmianę plików. Przed wysłaniem promptu naciskaj `Shift+Tab`, aż interfejs pokaże tryb Plan. Sprawdź widoczną nazwę trybu, nie liczbę naciśnięć. Jeśli skrót nie działa, sprawdź lokalne `/help`; nie zastępuj tego kroku samym dopisaniem „tylko plan" w zwykłym trybie.

Tryb Plan służy uzgodnieniu rozwiązania przed zmianą kodu. Prompt poniżej dodatkowo ogranicza zakres: odczyt wskazanych źródeł, bez testów i poleceń. Tryb nie zastępuje tych ograniczeń ani zgód na narzędzia. Nie wybieraj proponowanego rozpoczęcia implementacji. Ewentualny roboczy plan przechowywany przez CLI w danych sesji nie jest jeszcze plikiem w twoim projekcie — zapiszesz go osobno w kroku 4.

```
Przeczytaj starter\brief.md (kontrakt początkowy i nowe wymaganie),
triage\contracts.py, triage\service.py, tests\test_api.py
oraz tylko AC-04 i AC-05 ze starter\quality-standards.md.
Przygotuj plan testów dla odpowiedzi dostawcy:
category=other + actionability=actionable.
Wynik API ma zachować category=other, ale otrzymać
actionability=unclear, review_status=manual_review
oraz jira_draft=null.

Zwróć osobno:
1. Stan obecny: fakty z plików, z nazwami symboli/testów.
   Oddziel odczytane asercje od wyników wykonanych testów.
2. Wymaganie: reguła z briefu i właściwe AC, nie opis gotowego kodu.
3. Planowane asercje: przygotowanie atrapy, żądanie, status HTTP,
   pola odpowiedzi i przyczyna oczekiwanego niepowodzenia przed zmianą.
   Sprawdź istniejące pokrycie, nie proponuj duplikatów bez powodu.
4. Niewiadome i propozycje: czego nie potwierdzają te źródła
   i co trzeba rozstrzygnąć przed implementacją.
Na końcu wskaż zastosowane instrukcje i ich konkretny wpływ na plan.
Jeśli potrzebujesz innego źródła, podaj ścieżkę i powód; nie otwieraj
go bez uzgodnienia. Tylko odczyt i odpowiedź: bez implementacji,
edycji plików projektu, delegacji, testów i poleceń powłoki.
```

3. Sprawdź odpowiedź, nie tylko jej nagłówki Najpierw rozwiń References / użyty kontekst i wywołania narzędzi. Sprawdź odczyt wybranych źródeł. Sama lista nazw w odpowiedzi nie potwierdza ich użycia. Gdy host nie pokazuje odczytów, nie zakładaj, że agent rzeczywiście się na nich oparł.

Następnie porównaj odpowiedź z poniższymi wskazówkami. Pomagają sprawdzić zgodność odpowiedzi ze źródłami; nie wklejaj ich agentowi zamiast źródeł.

Stan obecny: w niezmienionym starterze `Classification` i `TriageResult` nie mają `actionability`. `Classification.from_payload` wymaga dokładnego zestawu dotychczasowych pól, więc dodatkowe pole nie jest jeszcze obsługiwane. `triage_report` już kieruje kategorię `other` do `manual_review` bez szkicu. Agent ma wskazać te fakty w kodzie, a nie twierdzić, że istnieje już normalizacja nowego pola. Jeżeli twoja kopia jest inna, opis ma wynikać z jej bieżących źródeł.

Testy a wykonanie: `test_unclear_report_requires_manual_review` sprawdza `other`, status i brak szkicu, lecz nie sprzeczną odpowiedź z `actionability`. Nazwa `test_actionable_report_has_draft` również nie dowodzi obsługi tego pola — przeczytaj asercje. W C2 oczekuj zdania „testów nie uruchomiono". Wynik z wcześniejszego etapu można przywołać tylko z jego pochodzeniem i zakresem; nie potwierdza on przyszłego testu normalizacji.

Wymaganie: odpowiedź przypisuje normalizację do nowego wymagania w briefie i AC-04. Odróżnia dozwoloną kategorię `other` ze sprzeczną potrzebą działania od kategorii spoza kontraktu, dla której AC-05 nadal wymaga błędu `502`. Nie zmienia `other` na „bardziej pasującą" kategorię i nie wyprowadza decyzji wyłącznie z sentymentu.

Planowane asercje: plan powinien opisać następującą próbę:

1. Atrapa dostawcy zwraca `other` i `actionable` oraz poprawne pozostałe pola klasyfikacji. To odpowiedź dostawcy, **nie dodatkowe pola żądania użytkownika**. Nie potrzeba połączenia `live`.
2. Żądanie `POST /api/triage` zawiera syntetyczny `text` i `store_id=store-001`, zgodnie z obecnym wejściem API.
3. Po wdrożeniu oczekuj HTTP `200` i następujących wartości:

   ```
   category == "other"
   actionability == "unclear"
   review_status == "manual_review"
   jira_draft is None
   ```

Brak klucza `jira_draft` nie jest tym samym co jawne `null`; sam status HTTP nie wystarcza.

4. Plan wskazuje ochronę dotychczasowego kierowania do `manual-triage` oraz regresje AC-05, z wykorzystaniem istniejących testów tam, gdzie wystarczają. Nie porównuje swobodnego uzasadnienia modelu z jednym sztywnym zdaniem.

Oczekiwane niepowodzenie przed zmianą ma wynikać z braku obsługi nowego kontraktu, a nie błędu importu czy źle przygotowanego żądania. Wybrane pliki pokazują odrzucenie dodatkowego pola przez walidację; bez odczytu obsługi wyjątków lub wykonania testu agent nie powinien przedstawiać dokładnej odpowiedzi HTTP tej nowej próby jako zaobserwowanej. Docelowe HTTP `200` jest wymaganiem planu.

Niewiadome i propozycje: konkretny sposób rozszerzenia kontraktu i miejsce normalizacji są na tym etapie propozycją do uzgodnienia. Jakość rzeczywistego klasyfikatora i wynik przyszłego uruchomienia pozostają niepotwierdzone. Test z atrapą sprawdzi obsługę wyniku, nie zdolność modelu do rozpoznania sprawy. Nie akceptuj ogólnego „wszystko sprawdzone" ani dopisywania wymagań bez źródła.

Instrukcje: oczekuj przykładów wpływu zasad C1, np. syntetycznych danych i atrapy w planie testu oraz walidacji na granicy usługi. To zasady *jak pracować*, nie źródło biznesowej decyzji `unclear`. Zasada wykonywania testów po zmianie nie upoważnia do uruchamiania ich w próbie, która zabrania edycji i poleceń.

Jeśli odpowiedź miesza te kategorie, wskaż jedno konkretne zdanie: „Czy to fakt z pliku, wymaganie, propozycja czy wynik uruchomienia? Podaj źródło lub oznacz jako niepotwierdzone. Nadal bez zmian i poleceń". Jeśli agent zaczyna wdrażać funkcję, zatrzymaj go; nie akceptuj edycji jako wyniku tego kroku.

4. Przełącz tryb i zapisz plan lokalnie Po sprawdzeniu odpowiedzi naciśnij `Shift+Tab`, aby przejść z Plan do **zwykłego trybu interaktywnego**, nie Autopilot. Sprawdź nazwę trybu przed wysłaniem kolejnego promptu. Teraz zezwalasz na jedną konkretną zmianę: zapis dokumentu, **nie wykonanie planu**.

```
Zapisz sprawdzony wyżej plan testów jako Markdown
w plans\c2-actionability.md, względem katalogu głównego tej kopii.
Utwórz folder plans, jeśli nie istnieje. Jeśli plik już istnieje,
zapytaj przed nadpisaniem.
Zachowaj fakty ze źródłami, wymagania i AC, planowane asercje,
niewiadome oraz informację, że testów nie uruchomiono.
Zgoda dotyczy tylko tego folderu i pliku. Nie wykonuj planu,
nie zmieniaj kodu, testów ani konfiguracji. Nie uruchamiaj testów
ani poleceń powłoki. Po zapisie odczytaj plik i potwierdź jego ścieżkę.
```

Samodzielnie otwórz `plans\c2-actionability.md` w edytorze i porównaj go ze sprawdzoną odpowiedzią. Sama deklaracja „zapisano" nie wystarcza. Sprawdź, czy nie zmieniły się kod, testy i konfiguracja; zapis planu nie oznacza wykonania jego kroków. To twoja lokalna pomoc do dalszej pracy, nie plik dostarczony w starterze.

Wróć przez `Shift+Tab` **do Plan** i potwierdź nazwę trybu. Pozostań w nim podczas przygotowania notatki i dalszej analizy. W ten sposób rozdzielasz uprawnienia: planowanie nie daje zgody na implementację, a zgoda na zapis dokumentu nie daje zgody na zmianę aplikacji.

5. Przekaż plan do nowej sesji Po zapisaniu planu dodaj samodzielnie w edytorze skill triage-handoff do `.github\skills\triage-handoff\SKILL.md`. To osobne przygotowanie konfiguracji; nie jest zgodą na zmianę aplikacji. Skill tworzy odpowiedź w rozmowie, nie potrzebuje skryptu wykonawczego.

```
Użyj triage-handoff do przekazania planu testów other + actionable.
Odczytaj plans\c2-actionability.md; traktuj go jako plan, nie dowód
wykonania. Podaj jego ścieżkę w notatce.
Maksymalnie 250 słów: cel i AC, fakty z odnośnikami do źródeł,
decyzje, wykonane testy i wyniki, niewiadome, wyłączenia
oraz jeden następny krok i sposób jego sprawdzenia.
Nie przedstawiaj planowanych testów jako wykonanych.
Nie implementuj, nie zapisuj plików i nie uruchamiaj poleceń.
```

Oczekuj notatki do 250 słów, która zachowuje różnicę między obecnym kontraktem i wymaganiem docelowym oraz informację o niewykonaniu testów. Sprawdź wywołanie skilla i źródła twierdzeń. Nie przekazuj całej rozmowy.

Rozpocznij pustą rozmowę w tej samej kopii. Ponownie ustaw tryb Plan przez `Shift+Tab` i sprawdź jego nazwę. Wklej sprawdzoną notatkę z poniższym zleceniem:

```
Zweryfikuj poniższą notatkę na podstawie ponownego odczytu
wymienionych w niej źródeł oraz plans\c2-actionability.md.
Plan nie zastępuje bieżących źródeł. Wskaż twierdzenia potwierdzone,
nieaktualne i niepotwierdzone. Oddziel wymagania od gotowego kodu.
```

```
Zaproponuj tylko jeden następny krok; nie wykonuj go.
Bez edycji, testów i poleceń powłoki.
[Tutaj wklej notatkę.]
```

Sprawdź przed dalszą pracą: czy przełączyłeś Plan → zwykły tryb tylko na zapis → Plan i czy `plans\c2-actionability.md` zawiera sprawdzony plan, bez zmian aplikacji? Zobacz, czy nowa sesja ponownie czyta źródła, zamiast tylko potwierdzać notatkę. Porównaj jej opis stanu obecnego, wymagania, asercji, niewiadomych i instrukcji ze źródłami. Jeśli nie ma rozbieżności, agent nie musi wymyślać błędu. Plan zachowaj do etapu 4, żeby wykorzystać go przy nowej funkcji. Teraz przejdź do maskowania, nie do implementacji `actionability`.

W CLI `/context` pokazuje zajętość okna kontekstu, a `/usage` zużycie zasobów w sesji. Opcjonalne `/compact` skraca historię, ale nie zastępuje nowej rozmowy ani ponownego odczytu źródeł. Nie zapełniaj sztucznie okna; w krótkiej sesji nie trzeba skracać historii.

Zadanie aplikacyjne: naprawa maskowania

Start: punkt odniesienia z etapu 1 jest zachowany. To już praca nad `#iss002` i AC-01, nie nad polem z planu C2.

Tutaj zmieniamy sposób pracy: zakończ bieżącą pracę w rozmowie CLI i przejdź do edytora Visual Studio Code. To próba **podpowiedzi kodu podczas pisania** (*inline suggestions*), nie trybu Agent, czatu Edit ani zlecenia „napraw test". Sam wybierasz miejsce edycji, wpisujesz kod, oceniasz sugestie i uruchamiasz polecenia w terminalu. Nie wykonuj planu `actionability`.

Co dokładnie sprawdza czerwony test Otwórz `tests\test_privacy.py` i znajdź funkcję:

`test_spaced_phone_is_redacted_before_model_and_export`

Najpierw sam uruchom plik testów w terminalu VS Code:

```
.\scripts\test.ps1 -PytestArgs @("tests\test_privacy.py", "-q")
```

W niezmienionym starterze oczekuj jednego FAIL. W teście `RecordingClassifier` dziedziczy po `DemoClassifier`. Jego `classify` najpierw zapisuje otrzymany argument w `self.received`, a potem wywołuje zwykłą klasyfikację demonstracyjną przez `super()`. To lokalny „rejestrator wejścia", nie połączenie z prawdziwym modelem. Początkowe `received = ""` nie oznacza, że tekst jest pusty po wywołaniu.

Test wywołuje bezpośrednio usługę:

```
provider = RecordingClassifier()
result = triage_report(
    "Brak całej dostawy. Kontakt +48 500 600 700.",
    "store-001",
    provider,
)
```

Tekst zawiera problem biznesowy i syntetyczny numer ze spacjami; `store-001` to identyfikator sklepu, a `provider` to rejestrujący klasyfikator. Następnie test sprawdza cztery rzeczy, w tej kolejności:

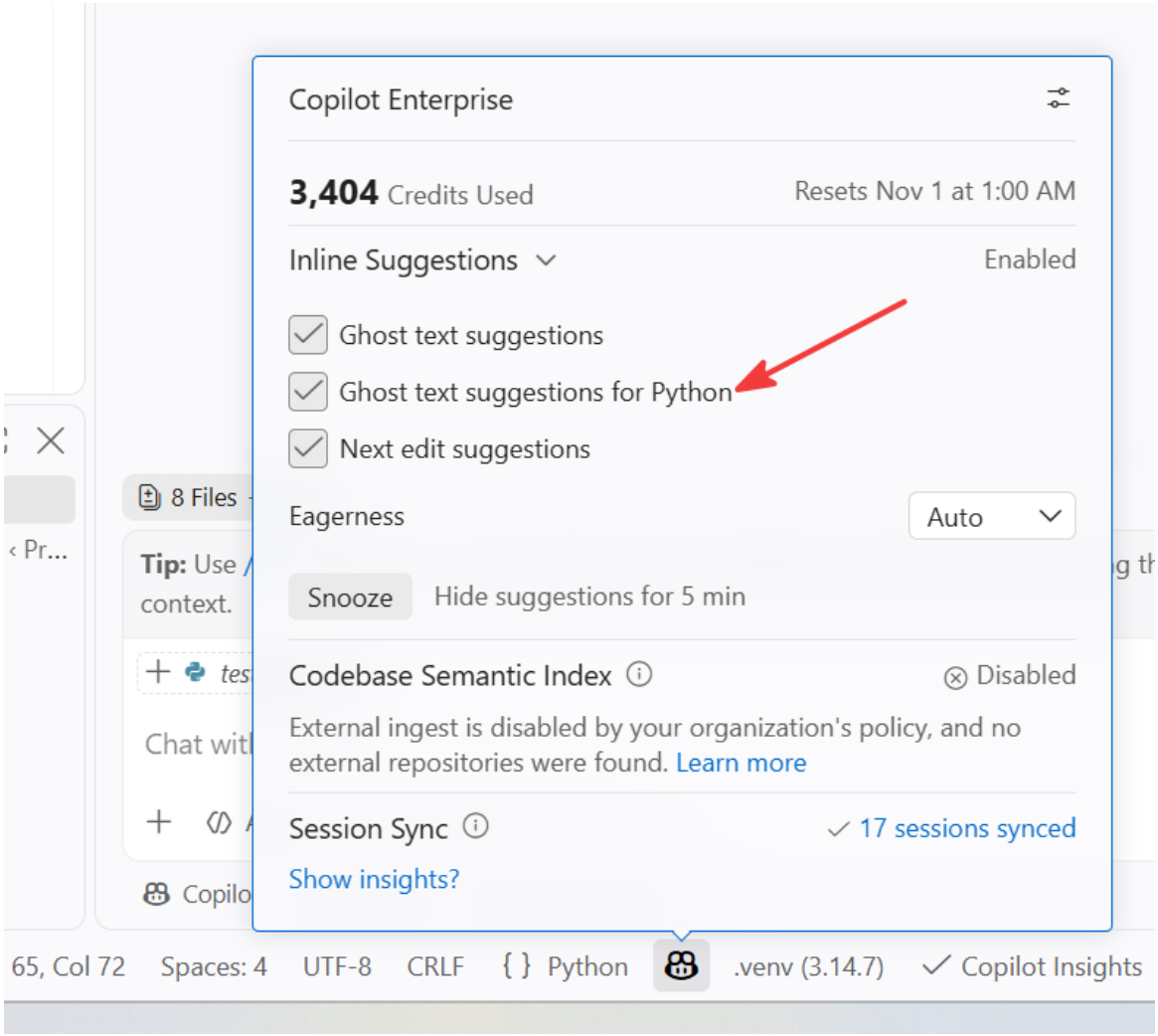
1. `provider.received` ma wynosić dokładnie `"Brak całej dostawy. Kontakt [PHONE]."`. Telefon musi być zamaskowany **zanim** klasyfikator dostanie tekst; reszta zdania ma pozostać.
2. `result["sanitized_text"]` ma być równe `provider.received`: usługa ma zwrócić ten sam oczyszczony tekst, który przekazała dostawcy. Sama równość nie dowodzi maskowania — temu służy pierwsza asercja.
3. `result["jira_draft"]` nie może być `None`. Zgłoszenie braku dostawy nadal ma prowadzić do utworzenia szkicu; ukrycie całego wyniku nie jest naprawą prywatności.
4. Opis szkicu nie może zawierać `"+48 500 600 700"`. Ta asercja sprawdza brak konkretnego numeru, nie pełną treść opisu ani wszystkie rodzaje PII.

Dlaczego FAIL? Starter maskuje telefon zapisany ciągiem cyfr, ale nie rozpoznaje wymaganego wariantu z odstępami między grupami cyfr. Dlatego klasyfikator otrzymuje `"Brak całej dostawy. Kontakt +48 500 600 700."` zamiast wersji z `[PHONE]`. Pierwsza asercja przerywa test; pozostałe nie są wtedy wykonywane. Ich obecność w pliku nie oznacza, że sprawdzenie szkicu już się powiodło.

To błąd obsługi formatu wejścia, nie jakości modelu ani konfiguracji usługi zewnętrznej. Nazwa „before model and export" opisuje granice prywatności: test obserwuje wejście lokalnego dostawcy i opis lokalnego szkicu. Nie wykonuje żądania HTTP, nie sprawdza ekranu i niczego nie wysyła do Jiry.

Naprawa przez pisanie w edytorze, nie przez agenta **Najpierw włącz podpowiedzi.** Otwórz plik Python i kliknij ikonę Copilota na dolnym pasku stanu VS Code. W sekcji **Inline Suggestions** zaznacz **Ghost text suggestions** oraz **Ghost text suggestions for Python** — tę drugą opcję wskazuje czerwona strzałka na obrazku. **Next**

edit suggestions możesz włączyć dodatkowo, aby otrzymywać propozycje kolejnych edycji. Nie klikaj **Snooze**, bo wstrzymuje podpowiedzi. Na zrzucie opcje są już zaznaczone.



Menu Copilota: włączenie podpowiedzi kodu dla Pythona.

1. Otwórz obok siebie test oraz `triage\service.py` i `triage\privacy.py`. Sam prześledź drogę tekstu: wejście usługi → oczyszczenie → argument klasyfikatora → wynik i szkic. Porównaj format telefonu z przypadkami, które już przechodzą. Wybierz fragment implementacji do edycji; w tym kroku nie korzystaj jeszcze z wariantu awaryjnego zamieszczonego po obu próbach podpowiadania.

2. W menu Copilota na pasku stanu VS Code sprawdź, czy podpowiedzi inline są włączone dla Pythona i nie są wstrzymane. Kliknij w kod, nie w pole czatu. Nad wybranym fragmentem możesz wpisać taki **roboczy komentarz opisujący zamiar**, a nie algorytm:

   ```
   # Maskuj wymagane formaty telefonu przed klasyfikacją i szkicem.
   # Zachowaj treść zgłoszenia, zwykłe ilości i identyfikator sklepu.
   ```

3. Rozpocznij własną edycję wybranego fragmentu i zatrzymaj się, by odczytać wyszarzoną propozycję (*ghost text*). Nie kopiuj rozwiązania z czatu. Porównaj sugestię z AC-01 i formatami z briefu: czy obejmuje przypadek ze spacjami, a zarazem zachowuje działające przypadki i treść biznesową? `Tab` przyjmuje widoczną sugestię, `Esc` ją odrzuca; `Ctrl+Right` pozwala przyjmować wyszarzony tekst po jednym słowie przy domyślnych skrótach Windows.

4. Jeśli nie pojawia się sugestia, sprawdź stan Copilota, doprecyzuj komentarz lub dopisz początek własnej zmiany. Podpowiedź nie jest gwarantowana i nie musi być poprawna; możesz dokończyć ręcznie. Opcjonalne *next edit suggestions* mogą zaproponować zmianę istniejącego kodu: najpierw obejrzyj różnicę, potem ją przyjmij. Nie zastępuj tej próby uruchomieniem agenta.

5. Zapisz plik i sam ponów testy prywatności. Jeśli nadal jest FAIL, porównaj wartość otrzymaną z oczekiwaną i wróć do edytora. Napraw implementację, nie oczekiwania testu; bez `skip` i `xfail`. Usuń roboczy komentarz, jeśli tylko powtarza już czytelny kod.

Druga próba podpowiadania — regresje. W `tests\test_privacy.py` zacznij własny nowy test, np. od poniższego komentarza i nagłówka, a ciało dopisuj z podpowiedziami:

```
# Sprawdź wejście dostawcy i szkic dla telefonu z myślnikami.
def test_dashed_phone_is_redacted_before_model_and_export():
```

Przed przyjęciem sugestii przeczytaj jej dane i asercje. Odrzuć sprawdzanie wyłącznie `redact(...)` albo samego istnienia szkicu: ta regresja ma obserwować również argument dostawcy. Dodaj też przypadki `500 600 700`, e-maila obok telefonu i zwykłej liczby sztuk; zachowaj kontrolę identyfikatora sklepu. Możesz rozszerzyć własny test o parametryzację. Nie osłabiaj pierwotnego testu, aby dopasować go do sugestii.

Wariant awaryjny — cały plik po poprawce Zajrzyj tutaj dopiero po obu próbach edycji z podpowiedziami. Jeśli utkniesz albo chcesz porównać wynik, poniżej jest przykładowa pełna zawartość `triage\privacy.py` po naprawie. To publiczny wariant pomocniczy do tego kroku; twoja poprawka może wyglądać inaczej. Oryginalny plik w paczce startowej nadal zawiera celowy błąd.

```
import re


EMAIL = re.compile(r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b")
PHONE = re.compile(
    r"(?<!\w)(?:\+48[ -]?)?\d{3}[ -]?\d{3}[ -]?\d{3}(?!\w)"
)
PESEL = re.compile(r"(?<!\w)\d{11}(?!\w)")


def redact(text: str) -> str:
    text = EMAIL.sub("[EMAIL]", text)
    text = PHONE.sub("[PHONE]", text)
    return PESEL.sub("[PESEL]", text)
```

Ten wariant dopuszcza spację lub myślnik między trzema grupami cyfr telefonu i zachowuje format bez separatorów oraz opcjonalny prefiks `+48`. Pozostałe maskowanie pozostaje bez zmian. Jest to nadal heurystyka dla formatów z briefu, nie uniwersalny detektor danych osobowych.

Porównaj plik ze swoją wersją w VS Code; jeśli korzystasz z przykładu, przenieś potrzebną zmianę samodzielnie w edytorze i zapisz plik. Nie zastępuj przy tym własnych regresji ani nie zmieniaj oczekiwań pierwotnego testu. Samo skopiowanie przykładu nie dowodzi poprawności — wykonaj poniższe sprawdzenie.

Wariant awaryjny — test telefonu z myślnikami Także ten przykład jest do użycia dopiero po własnej próbie w edytorze. Poniżej kompletna funkcja do dopisania w `tests\test_privacy.py`. Importy `DemoClassifier` i `triage_report` są już na początku tego pliku; nie trzeba dodawać ich ponownie. Jeżeli masz już funkcję o tej nazwie, porównaj i uzupełnij jej ciało zamiast tworzyć drugą definicję. Zachowaj pierwotny test telefonu ze spacjami.

```
def test_dashed_phone_is_redacted_before_model_and_export():
    class RecordingClassifier(DemoClassifier):
        received = ""

        def classify(self, text):
            self.received = text
            return super().classify(text)

    provider = RecordingClassifier()
    result = triage_report(
        "Brak całej dostawy. Kontakt 500-600-700.",
        "store-001",
        provider,
    )
    assert provider.received == "Brak całej dostawy. Kontakt [PHONE]."
    assert result["sanitized_text"] == provider.received
    assert result["jira_draft"] is not None
    assert "500-600-700" not in result["jira_draft"]["description"]
    assert "Kontakt [PHONE]." in result["jira_draft"]["description"]
```

Test rejestruje rzeczywisty argument dostawcy, porównuje go z dokładnym oczekiwanym tekstem i sprawdza wynik usługi. W szkicu wymaga nie tylko braku numeru, ale również zachowania fragmentu kontaktu z `[PHONE]`. Nie wystarczyłoby usunąć całego opisu. Na niezmienionym starterze oczekuj FAIL; po poprawce maskowania oczekuj PASS. To regresja jednego formatu, nie zastępstwo pozostałych przypadków: telefonu ze spacjami, e-maila obok telefonu, ilości i identyfikatora sklepu.

Jak sprawdzić: uruchom testy prywatności, a po poprawce cały zestaw:

```
.\scripts\test.ps1 -PytestArgs @("tests\test_privacy.py", "-q")
.\scripts\test.ps1
```

Obejrzyj brak wymaganych formatów PII zarówno w argumencie dostawcy, jak i szkicu oraz zachowanie zwykłych ilości i identyfikatora sklepu. Jeśli nadal jest FAIL, porównaj oczekiwaną i otrzymaną wartość w tym teście. Błąd importu oznacza problem uruchomienia, nie wykrycie regresji.

Sprawdź naprawę: uruchom pierwotny test i wszystkie regresje. Po naprawie powinno przechodzić wszystkich 71 testów dostarczonej aplikacji, a dodane regresje zwiększają tę liczbę. Szukaj zera błędów, nie dokładnie `71 PASS` w rozszerzonym zestawie. Jeśli test nadal wykrywa problem, wróć do poprawki przed kolejnymi zmianami.

Na koniec użyj `triage-handoff` ponownie, tym razem dla naprawy `#iss002`. Zachowaj limit 250 słów i sprawdź notatkę w nowej sesji jak w C2. Teraz podaj faktycznie wykonane testy i wyniki; nie przenoś informacji „testów nie uruchomiono” z wcześniejszej próby planowania.

Etap 3 — skille i refaktoryzacja K1

Opis zadania i kryteria: `#iss003`.

C3. Dwie procedury sprawdzania zmian

Start: masz własne instrukcje i zielone regresje po etapie 2. **Wynik:** dwa działające skille, nie tylko opisane procedury. Najpierw przeczytasz wzorce i zaplanujesz ich użycie albo napisanie własnych skilli, potem jawnie zlecisz przygotowanie i uruchomienie. Refaktoryzacja K1 jest kolejnym, osobnym krokiem.

Co oznacza `K1` i po co to robimy? To identyfikator pierwszego standardu kodu przyjętego w tym ćwiczeniu, opisanego w standardach jakości. Nie jest poleceniem Copilota ani nazwą skilla. W projekcie startowym `triage\providers.py` łączy reguły klasyfikacji trybu `demo`, prompt dla modelu, adapter SDK i wybór dostawcy. Reguły `demo` podejmują decyzje lokalnie, a adapter `live` odpowiada za komunikację z modelem językowym. To różne odpowiedzialności: zmiana sposobu łączenia z modelem nie powinna wymagać zmiany reguł klasyfikacji.

Celem `K1` jest rozdzielenie tych odpowiedzialności **bez zmiany tego, co zwraca aplikacja**. Nie ulepszasz teraz reguł, promptu ani metryk i nie dodajesz nowej funkcji. Tryb `demo` ma nadal działać bez tworzenia klienta LLM. Nie chodzi o rozbudowę architektury ani określoną liczbę klas czy plików, lecz o najmniejszą uzasadnioną zmianę organizacji kodu. Najpierw przygotujesz dwa skille jako powtarzalne narzędzia sprawdzania, a potem użyjesz ich przy refaktoryzacji. Oprócz zielonych testów porównasz klasyfikację i szkic każdego z tych samych 25 przypadków przed zmianą i po niej — sama identyczna metryka zbiorcza nie dowodzi braku zmian.

Przygotuj dwa katalogi:

- `.github\skills\triage-tests` z `SKILL.md` i `scripts\run.ps1`;
- `.github\skills\triage-metrics` z `SKILL.md` i `scripts\run.ps1`.

Każdy skrypt leży wewnątrz katalogu swojego skilla. Nie zastępuje głównych skryptów projektu w katalogu `scripts`.

Szybsza ścieżka — przeczytaj i wykorzystaj przykład. Nie musisz pisać obu procedur od zera. W bibliotece są kompletne, nieaktywne wzorce:

- Skill testów i jego skrypt wykonawczy;
- Skill metryk i jego skrypt wykonawczy.

Otwórz oba pliki `SKILL.md` oraz skrypty. Zauważ, że skill opisuje, kiedy i jak uruchomić procedurę oraz jak odczytać wynik, a skrypt wykonuje powtarzalne polecenia. Po poniższej próbie planowania skopiuj **całe katalogi** `triage-tests` i `triage-metrics` z `starter\copilot\skills` do `.github\skills` w swojej kopii. Sprawdź, czy pliki leżą bezpośrednio w miejscach podanych wyżej, bez dodatkowego poziomu `skills`. Jeśli katalog docelowy już istnieje, porównaj zawartość i zachowaj własne zmiany; nie nadpisuj go w ciemno. Biblioteka sama nie aktywuje skilli.

W zespole zwykle jedna osoba przygotowuje i utrzymuje takie wspólne skille, a potem pakuje je w plugin do użycia przez pozostałych. Nie każdy musi pisać identyczne skrypty od nowa — ważne, by umiał je przeczytać, uruchomić i ocenić wynik. **Pakowanie w plugin pokażemy później, w etapie 8.** Teraz pracujesz z lokalnymi `.github\skills`, bez instalowania pluginu.

Jedno zlecenie, dwie wersje W tym kroku przećwicz świadome doprecyzowanie zlecenia. Wybierz sposób startu: **napisz własny prompt** na podstawie wymagań zadania albo **skopiuj poniższy prompt**.

```
triage-tests ma uruchamiać pełny zestaw testów Pythona przez pytest,
używając interpretera projektu z .venv. Ma zapisywać raport wyników
testów wygenerowany przez pytest w formacie JUnit XML oraz rzeczywisty
kod wyjścia procesu pytest. triage-metrics ma mierzyć jakość
klasyfikatora regułowego w trybie demo, zapisywać raport metryk
w formacie JSON i kod wyjścia procesu ewaluacji oraz porównywać osobno
macro-F1 i priority_accuracy z 0.90, bez zaokrąglania.
Oba przyjmują nowy RunDirectory w reports\automation, nie nadpisują
wcześniejszych plików wynikowych i nie naprawiają aplikacji.
Niepowodzenie testów nie może blokować pomiaru.
```

1. Przygotuj wersję 1 promptu — własną albo skopiowaną. Dodaj na końcu: „Najpierw tylko plan i sposób weryfikacji, bez edycji i uruchamiania poleceń”. Wyślij ją agentowi. W szybszej ścieżce dopisz: „Przeczytaj wzorce `triage-tests` i `triage-metrics` wraz ze skryptami w `starter\copilot\skills`. Zaplanuj ich skopiowanie do `.github\skills` i sprawdzenie, zamiast generować nowe implementacje. Nie nadpisuj istniejących plików”.
2. Porównaj plan z wymaganiami powyżej. Wskaż konkretną lukę, niejednoznaczność albo element wymagający doprecyzowania. Jeśli plan jest poprawny, nie wymyślaj błędu: doprecyzuj np. sposób sprawdzenia, że istniejący raport nie zostanie nadpisany.

3. Dopisz do promptu własne, konkretne doprecyzowanie i ponów zlecenie planu w tej samej sesji. Porównaj odpowiedzi: co się zmieniło i czy doprecyzowanie pomogło.
4. Po doprecyzowaniu planu skopiuj wzorce albo jawnie zleć ich skopiowanie; jeśli wybrałeś pisanie od zera, zleć utworzenie skilli. Następnie wykonaj próby poniżej.

Porównaj prompty i odpowiedzi bezpośrednio w rozmowie.

JUnit XML to format raportu wyników testów Pythona generowanego przez pytest, nie framework Javy używany w ćwiczeniu. Opcja `--junitxml` zapisuje raport do `tests.xml`. Skrypt wykonawczy (runner) uruchamia testy lub pomiar i zapisuje ich wyniki; nie jest osobnym frameworkiem testowym. Znaczenie `macro_f1`, `priority_accuracy` i przykładowy odczyt raportu opisuje objaśnienie metryk.

W nowej sesji sprawdź `/skills list`, wywołanie `skill(...)`, skrypt wykonawczy i pliki:

```
Użyj triage-tests dla reports\automation\skill-tests-01.
Nie poprawiaj błędów.
```

```
Użyj triage-metrics dla reports\automation\skill-metrics-01.
Nie poprawiaj klasyfikatora.
```

Jeśli piszesz skille od zera, porównaj je z wzorcami w `starter\copilot\skills`. W obu ścieżkach przeczytanie albo skopiowanie pliku nie zastępuje próby wykonania. Wzorce w `starter\copilot\triage-workshop\skills` są wersją do późniejszego pluginu; nie aktywuj teraz drugiego zestawu o tych samych nazwach.

Jak rozpoznać wykonanie: obecność na liście skilli oznacza wykrycie, wywołanie `skill(...)` oznacza odczyt procedury, a dopiero uruchomiony skrypt i świeże pliki pozwalają sprawdzić, co się wykonało. Dla testów obejrzyj `tests.xml` i `tests-exit.json`; dla metryk `evaluation.json` i `metrics-exit.json` w katalogach wskazanych w zleceniach. Porównaj podsumowanie agenta z plikami i rzeczywistymi kodami procesów.

Po etapie 2 oczekuj zielonych testów. Metryki mogą jeszcze nie osiągać progów: poprawne zgłoszenie tego wyniku oznacza działający skill, nie zgodę na automatyczną poprawę klasyfikatora. Sprawdź odmowę ponownego uruchomienia tego samego skilla w katalogu z jego wcześniejszymi raportami, bez ich nadpisania. Jeśli skill jest niewidoczny, sprawdź folder projektu, `SKILL.md` i nową sesję; jeśli nie ma wyników, sprawdź wykonanie skryptu, ścieżkę i komunikat błędu. Nie twórz zastępczych raportów ręcznie.

Zadanie aplikacyjne: K1 bez zmiany zachowania

```
Przejrzyj kod wyłącznie względem K1. Najpierw zabezpiecz wyniki
klasyfikacji i szkicu dla tych samych 25 przypadków.
Następnie zaproponuj najmniejszą refaktoryzację oddzielającą
reguły demo od adaptera live. Nie poprawiaj reguł, promptu ani metryk.
Zachowaj publiczne importy albo uzasadnij zmianę.
Po mojej zgodzie wykonaj zmianę i pokaż wyniki przed/po.
```

Niezmieniona wartość macro-F1 nie wystarcza — porównaj wyniki każdego przypadku. Tryb `demo` nie może wymagać utworzenia klienta LLM. Testy charakteryzujące zapisują obecne zachowanie jako punkt odniesienia: przy K1 mają wykrywać niezamierzoną zmianę, a nie wymuszać nową funkcję. Przy porównaniu wyników użyj deterministycznego `demo` lub kontrolowanej odpowiedzi dostawcy; zmienność dwóch wywołań `live` nie dowodzi regresji.

Sprawdź sam: uruchom regresje i porównaj wyniki przed i po samej refaktoryzacji. Powinny być identyczne.

Gdy wynik się różni: wskaż konkretny przypadek i zmienione pole. Wyjaśnij różnicę przed przejściem dalej, nawet jeśli metryka wzrosła. Nie zastępuj pomiaru „przed” wynikiem „po”. K1 kończy się zmianą odpowiedzialności kodu, nie zachowania klasyfikatora.

Przykładowe podsumowanie refaktoryzacji w VS Code; nie zastępuje sprawdzenia zmian i wyników we własnej kopii:

Przejrzyj kod wyłącznie względem K1. Najpierw zabezpiecz wyniki klasyfikacji i szkicu dla tych samych 25 przypadków. Następnie zaproponuj najmniejszą refaktoryzację oddzielającą reguły demo od adaptera
live. Nie poprawiaj reguł, promptu ani metryk. Zachowaj publiczne importy albo uzasadnij zmianę. Po mojej zgodzie wykonaj zmianę i pokaż wyniki przed/po

10:00 PM

Completed 10 steps in 3m 7s

Wykonano zatwierdzoną refaktoryzację **wyłącznie K1**:

- `DemoClassifier` przeniesiony do demo.py, bez zmiany reguł.
- providers.py zachowuje publiczny import tej samej klasy, prompt, adapter live i fabrykę.
- Dodano testy charakteryzujące i uaktualniono dokumentację.

Sprawdzenie	**Przed**	**Po**
Przypadki demo	25	25
Szkice / brak szkicu	16 / 9	16 / 9
Pełne klasyfikacje i wyniki serwisu	zapis bazowy	**25/25 identyczne**
Regresje	72 PASS	**100 PASS**

Zbiory i prompt mają niezmienione SHA-256. Test niezależności demo od SDK przeszedł.

Pełne wyniki: przed / po.

Nie zmieniano metryk ani zastanych zmian. Nie uruchamiano live; bez commita i pushu. Zgodność przed/po potwierdza zachowanie, **nie poprawę jakości klasyfikatora**.

4 files changed +176 -40

10:31 PM • GPT-6.1 Sol • 31.7 credits

Rysunek 2: Przykładowe podsumowanie refaktoryzacji K1 w czacie VS Code.

Etap 4 — nowy kontrakt, cel i druga opinia

Opis nowej funkcji i kryteria: `#iss004`.

Start: zakończone K1 i porównanie 25 przypadków bez zmian. **Kolejność:** propozycje UI w C4 → plan `actionability` → krytyka C5 → twoja decyzja → test i implementacja. C4 przygotowuje wariant widoku do etapu 5; nie zleca jeszcze jego budowy.

C4. Cel i subagenci

W Copilot CLI najpierw sprawdź `/help`, `/usage` i dostępne opcje swojej wersji. Nie zakładaj, że interfejs VS Code ma te same komendy.

Cel tej próby: przyspieszyć pracę, zachowując kontrolę nad jej zakresem. Skopiuj gotowe prompty, zamiast projektować je od zera. Najpierw uzyskasz krótką listę potrzeb UI, potem równolegle trzy warianty ich realizacji. Nie uruchamiaj trzech pełnych implementacji: porównanie pomysłów jest tańsze niż scalanie konkurencyjnych zmian tego samego pliku.

`/goal`, `/autopilot` i `/fleet` — co się różni?

Polecenie	Znaczenie	Co ćwiczysz
`/goal <cel>`	Alias `/autopilot <cel>` w wersjach, które go obsługują	Określenie wyniku i warunku zakończenia
`/autopilot <cel>`	Ustawia cel pracy autonomicznej	Wykonanie ograniczonego zadania bez kolejnych „kontynuuj”
`/autopilot` bez argumentów	Przełącza tryb Autopilot	Kontrolę trybu przed kolejnym zleceniem
`/fleet <zadanie>`	Pozwala rozdzielić pracę między subagentów	Równoległe wykonanie niezależnych zadań

Nie ucz się fałszywego podziału „`/goal` planuje, `/autopilot` wykonuje”. `/goal` jest aliasem polecenia ustawiającego cel, nie trybem planowania. Różnica do przećwiczenia to **cel** („co ma powstać i kiedy skończyć”) oraz **tryb** („czy agent ma sam kontynuować kolejne kroki”). `/fleet` dotyczy jeszcze innej rzeczy: podziału pracy, nie kryterium sukcesu ani zgody na edycję.

Alias `/goal` wprowadzono w CLI `1.0.55`; pomoc lokalnego CLI `1.0.95` wymienia `/autopilot`, ale nie pokazuje osobno `/goal`. Brak aliasu na liście nie dowodzi jego niedostępności. Jeśli poniższa próba zwróci „nieznane polecenie”, zamień tylko `/goal` na `/autopilot`, zachowując treść i limit. Nie instaluj innej wersji tylko dla aliasu.

Próba A — ustaw cel przez `/goal` Wklej cały blok jako jedno zlecenie:

```
/goal Przygotuj maksymalnie 5 zmian UI do etapu 5.
Odczytaj starter\brief.md, starter\quality-standards.md
i triage\templates\index.html. Uwzględnij AC-06–AC-09.
Odróżnij obecne zachowanie od wymagań przyszłego actionability.
Tylko odczyt: bez edycji, poleceń, delegacji i wywołań live.
Zwróć tabelę: potrzeba użytkownika, proponowana zmiana,
AC, ryzyko, konkretna próba użytkownika.
Zakończ po tabeli; nie wdrażaj propozycji.
--max-ai-credits 5
```

Czego oczekiwać: do pięciu konkretnych propozycji związanych z kodem i wymaganiami, a nie ogólnej listy dobrych praktyk. Przykładowy poziom konkretu: „Oddziel ostatni wynik od stanu nowego żądania; AC-08; ryzyko pomylenia wyników; próba: sukces → kolejne żądanie → błąd”. To przykład formatu, nie wynik zaobserwowany w twojej aplikacji. To nadal propozycje: UI nie zostało zmienione, a testy nie były uruchamiane.

Próba B — przełącz Autopilot i wykorzystaj ten sam wynik Po próbie A sprawdź widoczny tryb. Jeśli Autopilot nadal jest włączony, wpisz samo `/autopilot` i potwierdź wyłączenie. Nie przełączaj go w ciemno: zakończenie celu nie musi oznaczać wyłączenia trybu. Samo przełączenie nie jest nowym zleceniem analizy.

Teraz włącz pracę z nowym, małym celem przez `/autopilot <cel>`. Nie generuj tej samej listy od początku:

```
/autopilot Sprawdź tabelę z poprzedniej odpowiedzi.
Na podstawie odczytanych wymagań wybierz 3 najważniejsze potrzeby UI.
Dla każdej podaj: decyzję użytkownika, AC i sposób sprawdzenia w aplikacji.
```

```
Jeśli brakuje źródła, odczytaj tylko potrzebny fragment.
Bez edycji, poleceń, delegacji, live i nowych wariantów.
Zakończ po 3 punktach; nie rozpoczynaj implementacji.
--max-ai-credits 5
```

Czego oczekiwać: krótkiej selekcji z uzasadnieniem i sposobami samodzielnego sprawdzenia zachowania aplikacji. Obie formy z celem powinny służyć temu samemu mechanizmowi pracy; nie oczekuj innej „inteligencji" po zmianie aliasu. Różna treść odpowiedzi nie dowodzi różnicy trybów.

Kredyty AI mierzą rozliczane zużycie usługi, tokeny opisują ilość przetwarzanego tekstu, pamięć podręczna oznacza ponowne wykorzystanie wcześniej przetworzonego wejścia, a okno kontekstu ma ograniczoną pojemność. Sprawdź `/usage`; nie traktuj limitu jako gwarancji maksymalnego kosztu. Każda próba i delegacje mogą zużywać kolejne kredyty; limit jednego celu nie jest wspólnym budżetem całego etapu. Po próbie B ponownie sprawdź tryb i wyłącz Autopilot, jeśli pozostał aktywny.

Próba C — trzy niezależne warianty przez `/fleet`

```
/fleet Przygotuj równolegle 3 warianty UI do etapu 5.
Deleguj po jednym wariancie do 3 subagentów, bez dalszych delegacji:
A - karta decyzji, B - lista kontrolna, C - oś czasu.
Każdemu przekaż 3 potrzeby z poprzedniej odpowiedzi oraz źródła:
starter\brief.md, starter\quality-standards.md
i triage\templates\index.html. Każdy ma je odczytać.
Każdy wariant ma uwzględniać AC-06-AC-09 oraz trzy stany:
szkic do zatwierdzenia, brak działania, ręczna weryfikacja.
Wszyscy pracują tylko odczytowo: bez edycji, poleceń i live.
Każdy zwraca maksymalnie 150 słów: układ, powiązanie z AC,
ryzyko i konkretne próby użytkownika. Nie przedstawiaj propozycji
jako zachowania już sprawdzonego w aplikacji.
Po otrzymaniu wyników porównaj warianty w jednej krótkiej tabeli.
Wskaż rekomendację i kompromis. Nie implementuj; czekaj na mój wybór.
```

Czego oczekiwać: trzech rzeczywistych delegacji i syntezy ich wyników, nie trzech nagłówków napisanych przez jednego agenta. Sprawdź wykonania i ich wyniki w `/tasks`. Samo `/fleet` nie gwarantuje, że każde zadanie zostanie rozdzielone; tutaj prompt jawnie definiuje trzy niezależne zlecenia. Jeśli brakuje delegacji, poproś o wykonanie brakującej części albo jawną informację o ograniczeniu, nie o udawany raport. Równoległość może skrócić czekanie, ale przy tak małej próbie narzut subagentów może przeważyć. Nie utożsamiaj większej liczby agentów z większą szybkością lub jakością.

Wybierz jeden wariant i jednego agenta odpowiedzialnego za edycję. Osobne konteksty subagentów nie oznaczają osobnych katalogów roboczych. Możesz użyć gotowej odpowiedzi, zmieniając wybór i uzasadnienie:

```
Wybieram A - kartę decyzji, bo najpierw pokazuje, czy trzeba działać.
Główny agent będzie jedynym wykonawcą zmian.
Zachowaj ten wybór w rozmowie na etap 5.
Potwierdź wybór i czekaj na osobny prompt planistyczny; bez edycji.
Następnie zaplanujemy dodanie pola actionability, które określa,
czy zgłoszenie wymaga działania, jest tylko informacją,
czy wymaga wyjaśnienia. Nie zaczynaj jeszcze planowania ani budowania UI.
```

Co dalej i gdzie będzie plan? Powyższa odpowiedź jedynie zachowuje wybór UI w bieżącej rozmowie; nie tworzy planu ani pliku. Teraz wyślij prompt z podrozdziału Zadanie aplikacyjne: `actionability`. Agent ma opisać, jak dodać pole potrzeby działania do kontraktu i obsługi zgłoszenia, kiedy przygotować szkic, kiedy go nie tworzyć i jak sprawdzić te zachowania testami. To plan zmian aplikacji, nie plan wyglądu karty. **Plan otrzymasz w odpowiedzi agenta w tej samej rozmowie** — bez zapisu nowego pliku i bez nadpisywania `plans\c2-actionability.md`. Plik z C2 jest wcześniejszym materiałem wejściowym, który trzeba zweryfikować po refaktoryzacji K1. Po otrzymaniu aktualnego planu przejdź do C5 i zleć jego krytykę; implementacja wymaga osobnej zgody.

Oczekiwany wynik C4: do pięciu propozycji, selekcja trzech potrzeb, trzy niezależne warianty, które porównasz pod kątem własnych potrzeb. Potwierdź brak nowych zmian plików względem początku C4. Jeśli opcja nie jest dostępna w twojej wersji, ustal, czego nie możesz w niej wypróbować. Nie usuwaj ograniczeń, aby wymusić wykonanie.

Zadanie aplikacyjne: plan `actionability`

Najpierw uzyskaj plan, następnie przejdź do jego krytyki w C5. Jeśli masz już aktualny plan w tej rozmowie, nie generuj go ponownie.

```
Zaplanuj rozszerzenie o actionability: actionable, informational,
unclear. Oprzyj się na briefie. Wskaż kontrakt, obu dostawców,
walidację, serwis, API, UI i testy AC-02-AC-05.
Uwzględnij plan C2 z plans\c2-actionability.md,
ale sprawdź go względem bieżącego kodu po K1. Zaplanuj nowy test
nieprzechodzący z właściwej przyczyny. Teraz tylko plan, bez edycji,
testów i poleceń; przekażę go do krytyki przed decyzją o implementacji.
Nie twórz integracji z zewnętrznym systemem zgłoszeniowym.
Zwróć krótki plan w kolejności zależności, listę miejsc do zmiany
i tabelę: scenariusz, oczekiwane pola, test.
Umieść plan wyłącznie w odpowiedzi w tej rozmowie.
Nie zapisuj pliku ani nie nadpisuj plans\c2-actionability.md.
Oddziel minimalną obsługę nowego pola w UI od przebudowy widoku
w etapie 5.
```

Czego oczekiwać po planie: wskazanych miejsc w bieżącym kodzie, kolejności zmian i testów konkretnych wartości, nie samego HTTP 200. Plan powinien wykorzystać ustalenia C2, ale nie zakładać, że po K1 kod nadal leży w tych samych miejscach. Teraz przejdź do C5 i przekaż ten plan krytykowi. Nie akceptuj automatycznego przejścia do wykonania.

C5. Krytyka planu

Przekaż plan z poprzedniego kroku Rubber Duck, czyli agentowi pełniącemu rolę krytyka. Jego zadaniem jest wskazanie ryzyk i braków, nie wykonanie zmian. Nadal pracuj w tej samej rozmowie:

```
/rubber-duck Sprawdź plan actionability z poprzedniej odpowiedzi.
Odczytaj starter\brief.md, starter\quality-standards.md
i wskazane w planie miejsca kodu. Zakres: AC-02-AC-05.
Zweryfikuj pochwałę/no_action, other+actionable,
other+informational, kolejność walidacji i normalizacji oraz awarię dostawcy.
Dla istotnej uwagi podaj: punkt planu, scenariusz, źródło wymagania,
ryzyko i proponowany test. Oddziel braki planu od pytań.
Tylko krytyka: bez edycji, poleceń i testów. Zakończ po opinii.
```

Porównaj każdą istotną uwagę z wymaganiami i kodem; przyjmij ją albo odrzuć, podając uzasadnienie. Druga opinia sama w sobie nie potwierdza poprawności.

Jak sprawdzić C5: przy każdej istotnej uwadze znajdź scenariusz, źródło wymagania i proponowany sposób sprawdzenia. Oczekuj krytyki konkretnego planu, nie ogólnych porad. Krytyk może nie zgłosić uwag; nie wymuszaj znalezienia błędu ani nie traktuj braku uwag jako dowodu poprawności. Jeśli krytyk nie otrzymał planu, przekaż go i ponów tylko opinię.

Zadanie aplikacyjne: wykonanie uzgodnionego planu

Po C5 i sprawdzeniu planu jawnie zleć jego wykonanie: najpierw dodanie i uruchomienie testu nowego wymagania, potem implementację i regresje. Tutaj kończy się tryb „tylko plan”. Nie oczekuj, że test nowej funkcji był już częścią startowych 71 testów.

Prompt do wykonania po twojej decyzji. Najpierw odnieś się do uwag krytyka: napisz, które przyjmujesz lub odrzucasz i dlaczego. Jeżeli nie było uwag, nie wymyślaj ich. Następnie wyślij:

```
Wykonaj uzgodniony plan actionability z uwzględnieniem mojej decyzji po C5.
Jesteś jedynym wykonawcą zmian; bez równoległych edycji.
Najpierw dodaj i uruchom test nowego wymagania.
Potwierdź, że nie przechodzi z powodu zachowania, nie błędu importu.
Potem wdroż minimalną zmianę we wszystkich miejscach planu
i uruchom powiązane testy oraz regresje.
Zachowaj AC-01, podział odpowiedzialności K1 i jawne błędy 502,
bez zastępowania błędu wynikiem demo.
Nie przebudowuj jeszcze UI z etapu 5 ani nie poprawiaj ogólnie metryk.
Bez nowych pakietów i integracji z systemem zgłoszeniowym.
Na końcu podaj: co zmieniono, rzeczywiste polecenia i wyniki,
```

`które scenariusze sprawdzono oraz co pozostało niesprawdzone.`
`Przy blokadzie zatrzymaj się i opisz ją; nie deklaruj sukcesu.`

Czego oczekiwać po wykonaniu: rzeczywistego przejścia nowego testu z czerwonego do zielonego i wyników regresji, nie tylko opisu zmian. Lista niesprawdzonych rzeczy podpowiada, czym zająć się dalej. Wybrany w C4 układ nadal czeka na etap 5.

Sprawdź nową funkcję: porównaj pola dla czterech przykładów z briefu oraz sprzeczności `other + actionable` i `other + informational`. Kategoria `other` ma pozostać bez zmian, a potrzeba działania przyjąć `unclear`, ze statusem `manual_review`. Nie wyprowadzaj tej decyzji wyłącznie z sentymentu. Zobacz, czy zwracają oczekiwany szkic albo jawne `null`, a regresje awarii i niepoprawnej kategorii nadal dają `502` bez szkicu i zastępczego sukcesu `demo`. Sprawdź testy, nie tylko deklarację agenta. Atrapa potwierdza obsługę wyniku; przykłady AC-03 sprawdź również na używanym klasyfikatorze. Pełne dopracowanie widoku wykonasz w etapie 5.

Etap 5 — użyteczny interfejs i K2

Opis problemów interfejsu i kryteria: `#iss005`.

Start: API obsługuje trzy ścieżki decyzji z etapu 4. Przypomnij sobie wybrany wariant UI z C4. Jeśli API zwraca błędne pola, wróć do etapu 4 — nie ukrywaj problemu przez zmianę samego widoku.

Użyj lokalnych zasad z `triage\templates\AGENTS.md`; nie twórz drugiej, sprzecznej konfiguracji.

K2 oznacza rozdzielenie wysyłania żądań HTTP od prezentowania wyniku. Najpierw uporządkuj te odpowiedzialności bez zmiany zachowania i sprawdź wynik. Następnie wprowadź uzgodnione zmiany widoku. Dzięki temu odróżnisz regresję po refaktoryzacji od celowej zmiany interfejsu.

```
Przeczytaj U1-U3, K2 i AC-07-AC-09. Najpierw określ, co użytkownik ma
rozumieć z interfejsu. Wykorzystaj wariant wybrany w C4.
Najpierw rozdziel HTTP i prezentację bez zmiany zachowania
i sprawdź tę refaktoryzację. Dopiero potem popraw widok.
Pokaż po polsku decyzję, kategorię, priorytet, zespół,
krótkie uzasadnienie i następny krok.
JSON pozostaw w zwijanych szczegółach. Oddziel stan żądania
od ostatniego wyniku. Zachowaj tekst po błędzie i umożliw ponowienie.
Oddziel obsługę żądań HTTP od wyświetlania wyniku.
Zachowaj etykiety, aria-live, blokadę wielokrotnego wysłania
i textContent. Bez nowych pakietów i zmian kontraktu API.
```

Sprawdź trzy stany: „Szkic do zatwierdzenia”, „Brak działania” i „Wymaga ręcznej weryfikacji”. W każdym stanie ma być jasne, że niczego nie wysłano.

Bez otwierania JSON odczytaj decyzję, priorytet, zespół i stan wysłania. Sprawdź sekwencję sukces → nowe żądanie → błąd → ponowienie: tekst pozostaje w formularzu, poprzedni wynik jest oznaczony jako poprzedni, a ponowienie daje aktualny wynik. Obsłuż formularz klawiaturą i sprawdź widoczny fokus; przy szerokości 375 px główny widok nie przewija się poziomo. Fokus to widoczne wskazanie elementu, na którym zadziała klawiatura. Przejdź między polami i przyciskiem przez `Tab`, wróć przez `Shift+Tab` i wyślij formularz bez myszy. Szerokość 375 px ustaw w narzędziach deweloperskich przeglądarki; nie chodzi o pomniejszenie strony zoomem.

Jak przygotować próbę: użyj pochwały, komentarza mieszanego i prośby o kontakt z briefu. Błąd HTTP zasymuluj lokalną atrapą dostawcy; nie wywołuj awarii wspólnej usługi. Osobno obejrzyj reakcję na brak połączenia. Ustal kontrolowany sposób próby przed jej wykonaniem. Po zmianach uruchom ponownie własny serwer i odśwież stronę, żeby nie sprawdzać starej wersji UI.

Wypróbuj widok samodzielnie: wykonaj opisane czynności w przeglądarce i sprawdź, czy rozumiesz wynik bez JSON. Odpowiedź agenta, test API ani zrzut ekranu nie potwierdzają klawiatury i ponowienia. Jeżeli agent nie mógł obsłużyć przeglądarki, wykonaj te próby sam; nie oznaczaj ich jako wykonanych na podstawie samego kodu.

Etap 6 — skill CSV, scenariusz aplikacyjny i pomiar

Opis zadań CSV, komentarza mieszanego i pomiaru: `#iss006`.

C6. Skill z podfolderami

Po co CSV w tym warsztacie? Wyobraź sobie plik z syntetycznymi kosztami warsztatu: nazwa pozycji, liczba sztuk i cena jednostkowa. Przed importem chcesz wiedzieć, czy dane spełniają ustalone reguły, gdzie są błędy i jaką wartość mają poprawne pozycje. Nie budujesz jednak importera ani nowej funkcji aplikacji Flask. To mały, niezależny przykład, na którym łatwo sprawdzić, czy Copilot wykonał procedurę, zamiast tylko napisać przekonującą odpowiedź.

Co dokładnie sprawdza walidator? Reguły opisuje kontrakt wejścia:

- format UTF-8, separator przecinkowy, nagłówek `item,quantity,unit_price` w tej kolejności i co najmniej jeden rekord;
- dokładnie trzy komórki w rekordzie oraz niepuste `item`;
- `quantity`: liczba całkowita od 1 do 9999, bez znaku i zer początkowych;
- `unit_price`: cena nieujemna, z kropką i najwyżej dwoma miejscami po przecinku, bez waluty i notacji wykładniczej. Dlatego `0.10` przechodzi, a `1e2` nie — liczy się kontrakt, nie samo znaczenie liczby.

Rekord oznacza tutaj jedną pozycję danych, bez wiersza nagłówka. W raporcie JSON zobaczysz liczbę rekordów poprawnych i błędnych, problemy wskazane przez numer rekordu i nazwę pola oraz `valid_total`: sumę `quantity × unit_price` **tylko dla poprawnych rekordów**, liczoną przez `Decimal`, czyli typ do dokładnych obliczeń dziesiętnych. Copilot ma przedstawić te wyniki według szablonu, bez kopiowania zawartości komórek. PASS oznacza zgodność z kontraktem, FAIL — wykryte błędy rekordów, a BLOCKED — przeszkodę w wykonaniu kontroli, np. błędny nagłówek lub zajętą ścieżkę raportu. Przy BLOCKED może nie powstać nowy JSON: trzeba pokazać komunikat i kod procesu, nie przedstawiać starego raportu jako nowego wyniku.

Jak to się ma do Copilota? W C3 korzystałeś ze skilli do testów i pomiaru aplikacji. Teraz sam opisujesz w `SKILL.md`, kiedy i jak agent ma wykonać inną powtarzalną pracę. `references` dostarcza reguły, `assets` — szablon odpowiedzi, `examples` — dane do prób, a `scripts` — gotowy walidator. Copilot zbiera ścieżki, korzysta z tych zasobów, uruchamia skrypt i wyjaśnia jego wyniki; **to skrypt, nie model językowy, rozstrzyga poprawność danych i oblicza sumę**. Ćwiczysz więc podział odpowiedzialności między instrukcję dla agenta, materiały pomocnicze i deterministyczne narzędzie.

Walidacja nie potwierdza zgodności z fakturami, nie sprawdza podatków ani kursów walut i niczego nie importuje. Nie mierzy też jakości klasyfikatora — temu służą dalsze części etapu 6.

Twoje zadanie. Utwórz `.github\skills\csv-quality`. Skopiuj podfoldery `references`, `assets`, `examples` i `scripts` z materiału, ale napisz własny `SKILL.md`.

```
Skill ma kontrolować syntetyczny CSV przed importem. Najpierw wymaga
ścieżek wejścia i nowego raportu, potem uruchamia lokalny walidator,
odczytuje JSON i rzeczywisty kod wyjścia. Rozróżnia PASS, FAIL i BLOCKED.
Korzysta z kontraktu, szablonu, przykładów i skryptu przez ścieżki względne.
Nie zmienia CSV, aplikacji ani instrukcji i nie instaluje pakietów.
```

Sprawdź `invalid.csv`, `valid.csv`, próbę nadpisania istniejącego raportu oraz własny przypadek z cenami `0.10` i `1e2`.

Jak zacząć: po utworzeniu skilla wywołaj go w nowej sesji, np.:

```
Użyj csv-quality. Wejście:
.github\skills\csv-quality\examples\invalid.csv
Nowy raport: reports\csv-invalid-01.json
Uruchom walidator, odczytaj wynik i kod wyjścia. Nie poprawiaj CSV.
```

Dla `valid.csv` wskaż osobny, nieistniejący raport. W próbie nadpisania podaj ponownie już wykorzystaną ścieżkę. Własny CSV przygotuj według kontraktu i szablonu z zasobów skilla: oba rekordy mają mieć poprawne pozostałe pola, żeby sprawdzić format ceny, a nie przypadkowy inny błąd.

Wejście	Oczekiwany wynik
`invalid.csv`	FAIL, kod wyjścia 1, 2/4 poprawne, suma `31.00`
`valid.csv`	PASS, kod wyjścia 0, 2/2 poprawne, suma `77.50`
istniejący raport	BLOCKED, kod wyjścia 2, bez nadpisania
ceny `0.10` i `1e2`	`0.10` poprawna; `1e2` odrzucona, mimo że opisuje liczbę

Na co zwrócić uwagę W wywołaniach narzędzi sprawdź odczyt kontraktu, szablonu i przykładów oraz uruchomienie walidatora. Porównaj kod wyjścia, liczniki i sumę w JSON z tabelą powyżej. Po próbie nadpisania sprawdź, że istniejący raport i wejściowy CSV pozostały bez zmian. Dla `0.10` i `1e2` porównaj wynik z kontraktem formatu ceny.

Oczekiwany FAIL dla `invalid.csv` oznacza poprawne wykrycie błędów danych, nie awarię skilla. Oczekiwany BLOCKED dla zajętego raportu chroni plik. Jeśli wynik jest inny, najpierw sprawdź ścieżki, odczytany kontrakt i rzeczywisty komunikat walidatora; nie zmieniaj wejścia, aby wymusić PASS.

Zadanie aplikacyjne: komentarz mieszany krok po kroku

Prześledź jeden konkretny komentarz: **„Miła obsługa, ale składanie zamówień nie działa."** Pochwała nie powinna przesłonić problemu z zamawianiem.

Oczekiwany wynik: `category = order`, `priority = P1`, `actionability = actionable`, `review_status = draft_ready` i niepusty szkic do zatwierdzenia. Szkic nie oznacza wysłania zgłoszenia.

1. **Przejdź do CLI.** Zatrzymaj edycje i zadania agenta w VS Code. W terminalu w katalogu swojej kopii projektu uruchom `copilot`. Pracuj na kodzie po etapach 1–5; nie cofaj wcześniejszych zmian.
2. **Znajdź sprawdzenie tego zachowania.** Przekaż poniższy prompt. Przeczytaj odpowiedź i wskazane asercje: sam status HTTP `200` nie wystarcza.

   ```
   Przeczytaj #iss006. Teraz zajmujemy się tylko komentarzem:
   „Miła obsługa, ale składanie zamówień nie działa."
   Znajdź testy tego scenariusza i porównaj je z AC-03 oraz briefem.
   Sprawdź: order, P1, actionable, draft_ready i niepusty szkic.
   Wskaż, co jest już pokryte, a czego brakuje. Oddziel test z atrapą
   dostawcy od sprawdzenia rzeczywistego klasyfikatora.
   Na razie bez edycji i uruchamiania poleceń.
   ```

3. **Uruchom test, uzupełniając tylko brakujące pokrycie.** Użyj następnego promptu. Jeżeli test już sprawdza wymagane zachowanie, nie twórz drugiego takiego samego.

   ```
   Uzupełnij istniejący test tylko o brakujące sprawdzenia tego
   scenariusza; dodaj nowy wyłącznie, jeśli nie ma odpowiedniego.
   Nie wywołuj zewnętrznego modelu w testach jednostkowych.
   Uruchom wybrane testy przez scripts\test.ps1 z -PytestArgs.
   Podaj polecenie, rzeczywisty kod wyjścia i wynik asercji.
   Nie zmieniaj jeszcze implementacji. Jeśli test zawiedzie,
   rozróżnij błąd wykonania od błędu zachowania, wskaż przyczynę
   i zaproponuj najmniejszą poprawkę. Zatrzymaj się przed jej wdrożeniem.
   ```

4. **Rozstrzygnij wynik testu.** Jeśli przechodzi, przejdź dalej — nie psuj działającego kodu, aby uzyskać czerwony test. Jeśli wykrywa błąd zachowania, przeczytaj diagnozę, zaakceptuj poprawkę i zleć ponowne uruchomienie tych samych testów. Nie pozwalaj osłabiać asercji ani dopasowywać reguł do ID lub całego tekstu przykładu. Błąd uruchomienia usuń przed oceną zachowania aplikacji.
5. **Sprawdź używany klasyfikator w aplikacji.** W działającym interfejsie z etapu 5 wklej dokładnie powyższy komentarz i wyślij go do analizy. Jeśli zmieniałeś kod, najpierw uruchom ponownie swój serwer. Zachowaj tryb aplikacji z pomiaru początkowego, a dla `live` również ten sam model. Porównaj wynik z pięcioma oczekiwaniami podanymi wyżej; wartości techniczne sprawdź w szczegółach JSON. Kategorię i priorytet oceń osobno. Test z atrapą nie dowodzi jakości używanego klasyfikatora. Rozbieżność przekaż agentowi do diagnozy i poprawki, a następnie powtórz tę samą próbę — nie zmieniaj komentarza, by uzyskać sukces. Jeśli zaczynałeś w `demo` i masz zatwierdzony dostęp do LLM, przed krokiem 6 wykonaj opcjonalny krok live.
6. **Sprawdź regresje i przejdź do pomiaru.** Uruchom `.\scripts\test.ps1` i sprawdź wynik oraz kod wyjścia. Wymagania obsługi błędów dostawcy z AC-05 nadal obowiązują jako regresje; nie są osobnym wariantem tego zadania. Po poprawnym wyniku przejdź do sekcji „Pomiar końcowy i interpretacja wyniku" poniżej. Ten jeden komentarz nie zastępuje pomiaru całego zbioru.

Krok z `demo` na rzeczywisty LLM przez Microsoft Entra To opcjonalna próba integracji po kroku 5. Możesz pozostać przy `demo`. W `demo` kategorię wybierają lokalne reguły. W `live` aplikacja wysyła zamaskowany komentarz do wdrożonego modelu przez Chat Completions. Microsoft Entra potwierdza tożsamość użytkownika, a uprawnienia zasobu decydują, czy może on wywołać model. Nie zmieniasz modelu Copilota — konfigurujesz osobny model językowy używany przez aplikację.

1. **Potwierdź dostęp i zatrzymaj swój serwer.** Organizator musi podać adres API v1 (endpoint), nazwę wdrożenia obsługującego Chat Completions, identyfikator dzierżawy Entra i zakres tokenu oraz potwierdzić uprawnienia

do wnioskowania, np. `Cognitive Services OpenAI User` na właściwym zasobie. Potrzebujesz dostępu sieciowego do endpointu i zgody na koszt wywołań. Samo konto Azure lub dostęp do Copilota nie wystarczają. W terminalu swojego serwera naciśnij `Ctrl+C`; pozostaw kod aplikacji i dane bez zmian.

2. **Sprawdź narzędzia.** W PowerShell w katalogu swojej kopii uruchom:

   ```
   az version
   .\.venv\Scripts\python.exe -c "import azure.identity, langchain_openai"
   ```

   Brak Azure CLI zgłoś organizatorowi; użyj zatwierdzonej firmowej procedury instalacji. Tylko jeśli brakuje bibliotek Pythona, uruchom:

   ```
   .\scripts\setup.ps1 -WithLLM
   ```

   Skrypt najpierw sprawdza dostępność zależności w już skonfigurowanym rejestrze lub proxy. Gdy nie są dostępne, zatrzymaj przygotowanie; nie przełączaj rejestru na publiczny.

3. **Ustaw lokalną konfigurację.** Samodzielnie utwórz lub uzupełnij `.env` w katalogu głównym swojej kopii, nie nadpisując innych ustawień. Poniższe URL i model są **przykładami do zmiany**, nie obietnicą dostępu do zasobu prowadzącego:

   ```
   TRIAGE_MODE=live
   OPENAI_AUTH_MODE=azure-cli
   OPENAI_BASE_URL=https://tkopaczfy27-6270-resource.services.ai.azure.com/openai/v1
   OPENAI_MODEL=gpt-6-luna
   OPENAI_TEMPERATURE=default
   OPENAI_API_KEY=
   AZURE_TENANT_ID=
   AZURE_TOKEN_SCOPE=https://ai.azure.com/.default
   ```

   Zamień `OPENAI_BASE_URL` na zatwierdzony adres API, a `OPENAI_MODEL` na **nazwę wdrożenia** na tym zasobie, która może różnić się od nazwy modelu. Wpisz w `AZURE_TENANT_ID` identyfikator dzierżawy od organizatora; pusta wartość korzysta z kontekstu Azure CLI, więc przy wielu dzierżawach wskaż ją jawnie. Zakres `https://ai.azure.com/.default` też musi odpowiadać konfiguracji endpointu. `OPENAI_API_KEY` pozostaw puste — używamy tokenu Entra, nie klucza. `default` pomija parametr temperatury w żądaniu, zamiast wymuszać `0`. Zmienne środowiskowe procesu mają pierwszeństwo przed `.env`; usuń stare nadpisania z konfiguracji własnego terminala, jeżeli je wcześniej ustawiałeś. Nie zakładaj, że edycja `.env` zmieni już uruchomiony proces.

4. **Zaloguj się własnym kontem przez Entra.**

   ```
   .\scripts\login.ps1
   ```

   Dokończ logowanie w przeglądarce, wybierając konto uprawnione do zasobu. Skrypt wywołuje `az login` dla skonfigurowanej dzierżawy i sprawdza pobranie tokenu. Gdy organizacja dopuszcza logowanie kodem urządzenia, alternatywą jest `.\scripts\login.ps1 -UseDeviceCode`. Oczekiwany komunikat zaczyna się od `PASS: Azure CLI token acquired`; nie oznacza jeszcze dostępu do endpointu ani poprawnego działania modelu. Logowanie nie nadaje ról.

5. **Uruchom aplikację jawnie w `live`.**

   ```
   .\scripts\run.ps1 -Mode live
   ```

   Pozostaw ten terminal uruchomiony. Samo `TRIAGE_MODE=live` w `.env` nie wystarczy przy użyciu skryptu: `run.ps1` bez `-Mode live` wybiera `demo`. Otwórz `http://127.0.0.1:5000` i odśwież stronę. W drugim terminalu sprawdź:

   ```
   Invoke-RestMethod http://127.0.0.1:5000/health
   ```

   Oczekuj `mode = live`. Pole `model_connectivity = not_checked` mówi wprost, że ten adres sprawdza serwer, **nie połączenie z LLM**.

6. **Wykonaj jedno rzeczywiste wywołanie.** W formularzu wybierz syntetyczny sklep, np. `store-001`, wklej komentarz z kroku 5 („Miła obsługa, ale składanie zamówień nie działa.") i uruchom analizę. To dopiero wysyła żądanie do modelu i zużywa jego limit. Obejrzyj wynik oraz szczegóły JSON: oczekujesz `order`, `P1`, `actionable`, `draft_ready` i szkicu. Poprawna odpowiedź potwierdza działanie tej próby, nie jakość modelu na całym zbiorze. Niepoprawna kategoria lub priorytet to problem jakości; błąd wywołania lub formatu odpowiedzi to problem integracji. Aplikacja ma pokazać jawny błąd, bez ukrytego powrotu do `demo`.

7. **Zachowaj porównywalność i wróć do kroku 6.** Jeżeli pomiar początkowy robiłeś w `demo`, końcowy pomiar rdzenia również wykonaj w `demo`. Po próbie zatrzymaj swój serwer przez `Ctrl+C`, przywróć `TRIAGE_MODE=demo` w lokalnej konfiguracji i, jeśli potrzebujesz UI, uruchom `.\scripts\run.ps1 -Mode demo`. Próba `live` pozostaje

osobną obserwacją. Jeśli chcesz dalej poprawiać jakość w `live`, najpierw utwórz **nowy punkt odniesienia dla tego trybu i wdrożenia**, przed kolejnymi poprawkami:

```
.\scripts\evaluate.ps1 -Mode live `
  -Output reports\baseline-live.json
```

Uruchamiaj go tylko w uzgodnionym oknie i limicie: domyślny zbiór to 25 wywołań modelu. Użyj nowej nazwy raportu, jeśli taki plik już masz. Nie porównuj go z `baseline-demo.json` jako pomiaru poprawy. Nie nazywaj pomiaru wykonanego teraz punktem odniesienia sprzed wcześniejszych zmian aplikacji.

Gdy próba jest zablokowana: problem logowania wymaga sprawdzenia konta, dzierżawy i zakresu; odmowa dostępu — uprawnień i sieci; brak wdrożenia — adresu i nazwy wdrożenia. Przy limicie lub przekroczeniu czasu oczekiwania uzgodnij ponowienie z organizatorem, nie uruchamiaj pętli żądań. Niepoprawny JSON lub nieobsługiwany parametr wymaga sprawdzenia zgodności modelu z adapterem. API aplikacji może zwrócić `502` zamiast kodu dostawcy; nie wyciągaj z samego `502` wniosku, że winne jest logowanie.

Pomiar końcowy i interpretacja wyniku

Co to jest `macro_f1`? `macro_f1` (macro-F1) mierzy **jakość przypisywania kategorii**. Porównuje kategorię przewidzianą przez klasyfikator z oczekiwaną w danych. Wynik mieści się od `0` do `1`; wyższy jest lepszy, a `1` oznacza bezbłędną klasyfikację kategorii na badanym zbiorze. Nie jest to pewność modelu ani po prostu procent poprawnych odpowiedzi.

Najpierw oblicza się F1 osobno dla każdej kategorii. Dla przykładowej kategorii `delivery` patrzymy na dwa rodzaje pomyłek: sprawy niesłusznie uznane za dostawę i prawdziwe sprawy dostawy przypisane gdzie indziej.

- **Precyzja (precision):** spośród spraw oznaczonych jako `delivery`, ile rzeczywiście dotyczy dostawy? Niska precyzja oznacza dużo spraw błędnie skierowanych do tej kategorii.
- **Czułość (recall):** spośród wszystkich spraw dostawy, ile klasyfikator rozpoznał jako `delivery`? Niska czułość oznacza pomijanie takich spraw.
- **F1:** średnia harmoniczna precyzji i czułości. Łączy oba aspekty; wysoki wynik wymaga, by oba były dobre.

Skrypt liczy F1 równoważnym wzorem opartym na liczbie trafień i pomyłek:

```
TP = poprawnie rozpoznane sprawy danej kategorii
FP = inne sprawy błędnie przypisane do tej kategorii
FN = sprawy tej kategorii błędnie przypisane gdzie indziej
F1 = 2 * TP / (2 * TP + FP + FN)
macro_f1 = suma F1 pięciu kategorii / 5
```

Przykład obliczenia, nie wynik twojego pomiaru: cztery trafienia, jedna błędnie dodana sprawa i jedna pominięta dają `F1 = 8 / (8 + 1 + 1) = 0.80`. Następnie, jeśli wyniki pięciu kategorii wynoszą `1.00`, `0.90`, `0.90`, `0.80`, `0.90`, ich `macro_f1` to `0.90`.

Dlaczego „macro”? Każda z pięciu kategorii ma równą wagę w średniej, niezależnie od liczby przykładów. Liczna, łatwa kategoria nie dominuje więc wyniku tylko dlatego, że występuje częściej. W naszym zbiorze rozwojowym i tak jest po pięć przykładów na kategorię. Średnia może jednak ukryć słabszą kategorię, dlatego obejrzyj też `per_category_f1` i konkretne `mismatches`. Próg `0.90` dotyczy średniej, nie każdej kategorii z osobna, i nie oznacza „90% zgłoszeń obsłużonych poprawnie”.

Czym różni się `priority_accuracy`? To zwykły odsetek przypadków, w których przewidziany priorytet zgadza się z oczekiwanym: `poprawne priorytety / wszystkie przypadki`. Na 25 przykładach `23 / 25 = 0.92` spełnia próg, a `22 / 25 = 0.88` nie. Można rozpoznać dobrą kategorię i zły priorytet albo odwrotnie, dlatego sprawdzasz **obie metryki osobno**.

Jak sprawdzić odpowiedź agenta: ma odczytać `mode`, `cases`, `macro_f1`, `priority_accuracy`, `per_category_f1` i `mismatches` z bieżącego raportu. Powinien powiedzieć osobno, czy pomiar się wykonał, czy osiągnięto każdy próg i gdzie są rozbieżności. Nie zaokrąglaj wartości w górę, żeby przekroczyły próg: `0.899` jest mniejsze niż `0.90`.

Te metryki nie mierzą `actionability`, poprawności szkicu, maskowania danych ani użyteczności UI. Wysokie F1 nie zastępuje testów tych wymagań. 25 jawnych przykładów to mały zbiór rozwojowy, nie dowód jakości produkcyjnej ani gwarancja zachowania na nowych danych.

Uruchom pomiary i porównaj je z punktem odniesienia Po sprawdzeniu komentarza mieszanego i ewentualnej zaakceptowanej poprawce wykonaj poniższe polecenia dla punktu odniesienia w `demo`:

```
.\scripts\evaluate.ps1 `
  -Mode demo -Output reports\final-demo.json -MinMacroF1 0.90
$LASTEXITCODE
.\scripts\evaluate.ps1 `
```

```
  -Mode demo -Dataset starter\data\challenge.json `
  -Output reports\challenge-demo.json
$LASTEXITCODE
.\scripts\test.ps1
$LASTEXITCODE
```

Jeśli masz osobny punkt odniesienia `live`, wykonaj pomiar w tym samym trybie i na tym samym wdrożeniu, zamiast powyższych dwóch ewaluacji:

```
.\scripts\evaluate.ps1 -Mode live `
  -Output reports\final-live.json -MinMacroF1 0.90
$LASTEXITCODE
.\scripts\evaluate.ps1 -Mode live `
  -Dataset starter\data\challenge.json `
  -Output reports\challenge-live.json
$LASTEXITCODE
```

Obie ewaluacje `live` wywołują model; uzgodnij limit dla 25 + 6 przypadków. `evaluate.ps1` bez `-Mode live` wybiera `demo` niezależnie od `.env`. Nie zastępuj pomiaru rdzenia w `demo` dodatkową próbą integracji `live`. `$LASTEXITCODE` odczytuj po każdym poleceniu: kolejny proces może zmienić tę wartość. Bezpośredni skrypt ewaluacji może nadpisać plik wskazany przez `-Output`; przy ponownym pomiarze wybierz nową nazwę raportu.

Sprawdź osobno macro-F1 >=0,90 oraz `priority_accuracy` >=0,90. Flaga `-MinMacroF1` sprawdza tylko pierwszy próg. Pomiar początkowy i wynik końcowy muszą używać tego samego trybu aplikacji i, dla `live`, tego samego modelu klasyfikującego. Nie chodzi o model wybrany w Copilocie; tryb `demo` nie używa modelu językowego.

Sprawdź jakość klasyfikatora: odczytaj obie metryki na 25 przypadkach i porównaj każdą z celem `0.90`, bez zaokrąglania. Raport kontrolny zawiera osobne sześć przypadków; obejrzyj rozbieżności, nie łącz zbiorów w jedną metrykę i nie dopisuj nieustalonego progu dla zbioru kontrolnego. Uruchom też testy funkcji i sprawdź ich wynik — F1 nie mierzy `actionability`. Jeśli polecenie nie ukończyło pomiaru, wyjaśnij błąd wykonania; jeżeli pomiar się udał, lecz próg nie został osiągnięty, analizuj błędne kategorie i priorytety. To dwa różne problemy.

Gdy nie osiągasz progu: wybierz rozbieżność z raportu i porównaj ją z regułą w briefie. Zleć wyjaśnienie przyczyny, a po ocenie propozycji wprowadź najmniejszą uzasadnioną poprawkę. Powtórz testy i pomiar na tym samym zbiorze oraz w tej samej konfiguracji. Nie poprawiaj etykiet ani reguł pod identyfikatory i pełne teksty przykładów. Jeśli użyjesz zbioru kontrolnego do dopracowania rozwiązania, jego kolejny wynik nie będzie już niezależnym sprawdzeniem na niewykorzystanych wcześniej danych.

Etap 7 — lokalny przegląd zmian

Opis przeglądu i kryteria: `#iss007`.

Przejrzyj zmiany z etapów 1–6, nie tylko ostatnią edycję. Jeśli korzystasz z Git, zacznij od stanu plików i zmian w katalogu roboczym:

```
git status --short
git --no-pager diff --stat
git --no-pager diff
git ls-files --others --exclude-standard
```

Jeśli utworzono tag `lab-start`, użyj go do porównania:

```
git --no-pager diff --stat lab-start
git --no-pager diff lab-start
```

`git diff lab-start` porównuje punkt startowy z bieżącą zawartością śledzonych plików: obejmuje łącznie zmiany zapisane w commitach, przygotowane i nieprzygotowane do commita. Nie pokazuje plików nieśledzonych — otwórz osobno każdy istotny plik z ostatniego polecenia. Pomiń pliki wynikowe; skup się na kodzie, testach i konfiguracji. Jeśli nie utworzono tagu, przejrzyj bieżące zmiany oraz istotne nowe pliki. Bez repozytorium Git porównaj swoją kopię z zachowaną kopią startową w edytorze. Nie inicjalizuj historii dopiero teraz, aby udawać, że obejmuje wcześniejsze etapy.

W CLI użyj `/review`, a w VS Code lokalnego Code Review, jeśli jest dostępny. Przekaż przygotowany diff i treść wskazanych nowych plików jako kontekst — sam opis zakresu nie gwarantuje, że narzędzie go odczyta:

```
/review Przejrzyj zmiany z etapów 1-6 i wskazane nowe pliki.
Jeśli przekazałem diff od lab-start, uwzględnij także zmiany już zapisane w commitach.
Potwierdź otrzymany zakres. Porównaj z AC-01-AC-10, K1 i K2;
szczególnie actionability z AC-02-AC-05.
Szukaj regresji kontraktu, złej normalizacji, utraty no_action i słabych
testów. Dla uwagi podaj plik, linię, scenariusz, wpływ, pewność i test.
Bez zmian, poleceń, GitHub i PR-a.
```

Sprawdź, czy narzędzie rzeczywiście objęło kod, który chcesz przejrzeć. Jeżeli lokalny przycisk lub `/review` obejmuje tylko zmiany robocze, użyj osobnej rozmowy z przygotowanym diffem i nowymi plikami do uzupełnienia zakresu. Ograniczonego przeglądu nie traktuj jako przeglądu całości.

Zweryfikuj istotne uwagi na podstawie kodu i odpowiedniego testu. Brak uwag nie zastępuje testów. Po poprawce ponownie uruchom odpowiednie regresje.

Następnie uruchom pełne testy (`.\scripts\test.ps1`), zatrzymaj własny serwer aplikacji i uruchom go ponownie w tym samym trybie. W drugim terminalu wykonaj `.\scripts\smoke.ps1` (ten sam port co serwer). Restart ani test kontrolny nie zastępują próby UI. Na koniec rzuć okiem, czy UI nadal działa, czy najważniejsze testy są zielone i czy zmiany odpowiadają zadaniom. Jeśli coś jest niejasne lub nie działa, wróć do tego fragmentu i sprawdź go ponownie po poprawce.

Sprawdź własny przegląd: czy porównałeś istotne uwagi z kodem i testami? Czy poprawki mają regresje, a test kontrolny po restarcie i próba UI dotyczą bieżącej wersji? Jeśli brakuje historii zmian, sprawdź, jaki zakres faktycznie objął przegląd, zamiast zakładać, że obejmował całość. Brak dostępu do narzędzia review można uzupełnić rozmową z rzeczywistym diffem, ale nie deklaracją „przejrzane” bez przekazania zmian.

Etap 8 — plugin do lokalnych sprawdzeń

Opis automatyzacji: `#iss008`.

Zbuduj `.copilot-plugins\triage-workshop` według wzorca:

```
triage-workshop\
  plugin.json
  agents\
    triage-conductor.agent.md
    triage-test-worker.agent.md
    triage-metrics-worker.agent.md
  skills\
    triage-tests\
    triage-metrics\
  scripts\
    summarize.ps1
    summarize.py
    test_summarize.py
```

Przenieś własne skille do pluginu, a aktywne duplikaty z `.github\skills` przenieś poza katalogi, z których Copilot wczytuje skille. Instrukcje projektu pozostają w `.github`. Koordynator deleguje dwa niezależne zadania i uruchamia podsumowanie dopiero po obu wynikach.

Sprawdź lokalne ładowanie:

```
copilot --plugin-dir .\.copilot-plugins\triage-workshop plugin list
copilot --plugin-dir .\.copilot-plugins\triage-workshop `
  --disable-builtin-mcps
```

Drugie polecenie otwiera sesję interaktywną. Po sprawdzeniu ładowania zakończ ją przed kolejnym poleceniem, aby nie uruchomić dwóch sesji wykonujących to samo zadanie. Nazwy agentów mają prefiks `triage-workshop:`, który wskazuje plugin. Uruchom jedno zlecenie:

```
$prompt = @'
Uruchom lokalne sprawdzenie po zmianach.
Użyj obu agentów pluginu i obu skilli, zbierz testy oraz metryki demo
i pokaż krótkie podsumowanie. Nie zmieniaj kodu. Wskaż ograniczenia.
'@
copilot --plugin-dir .\.copilot-plugins\triage-workshop `
  --agent triage-workshop:triage-conductor `
  --disable-builtin-mcps -i $prompt
```

Skrypt tworzy lokalne pliki wynikowe potrzebne do działania automatyzacji:

- `tests.xml` i `tests-exit.json`;
- `evaluation.json` i `metrics-exit.json`;
- `summary.json`.

Kod podsumowania: `0` = testy przeszły i oba progi zostały osiągnięte, `1` = FAIL, `2` = BLOCKED. Kod `0` procesu Copilot nie oznacza kodu `0` podsumowania. To lokalny wynik działania pluginu.

Oczekiwany wynik po etapie 7: testy i oba progi powinny dać PASS, ale agent ma odczytać bieżące pliki, a nie założyć sukces na podstawie poprzedniego etapu. Każdy przebieg używa nowego `RunDirectory`. FAIL testów nie może zatrzymać pomiaru; podsumowanie następuje po obu zadaniach, nie po pierwszym udanym.

Sprawdź działanie pluginu

Sprawdź, czy widzisz obie delegacje, uruchomienie obu skilli, testy oraz pomiar metryk. Obejrzyj `summary.json`: powinien odróżnić wynik testów i oba progi metryk od kodu wyjścia samej sesji Copilot. Na niezmienionym projekcie startowym automatyczne sprawdzenia powinny wykazać znany FAIL, bez automatycznej naprawy. Plugin nie zastępuje próby interfejsu ani testów właściwych dla funkcji.

Jeśli w podsumowaniu widzisz `acceptance: NOT_VERIFIED`, oznacza to, że automatyzacja nie potwierdziła całej akceptacji aplikacji. Nie jest to sprzeczne z PASS testów i metryk: każde z tych pól opisuje inny zakres.

Gdy coś nie działa: przy niewidocznym agencie sprawdź `--plugin-dir`, nazwę `triage-workshop:triage-conductor` i nową sesję. Przy złym skillu sprawdź aktywne duplikaty. Przy BLOCKED odczytaj konkretny brak lub odmowę uprawnienia; świeży `summary.json` może wtedy nie powstać. Nie używaj starego raportu jako wyniku nowej próby i nie nadawaj szerokich uprawnień tylko po to, aby otrzymać PASS.

Ćwiczenia dodatkowe

Spec Kit

Opcjonalne zadanie: `#iss010`, a po nim `#iss009`.

Wybór modelu Copilota: autor przećwiczył tę ścieżkę z GPT-6.1 Sol. Mniejsze modele, np. Luna, również można wykorzystać, lecz mogą wymagać bardziej szczegółowych poleceń i częstszej kontroli wyniku. Nie ma obowiązku wybierania większego modelu. Kieruj się dostępnością, kosztem i jakością odpowiedzi, a nie samą nazwą; wskazany model nie jest gwarancją poprawnego wykonania.

Wykonuj dopiero po etapach 1–8. Najpierw odtworzysz **stan istniejący**, a dopiero w następnym zadaniu opiszesz nową funkcję. Rozpoznaj całe rozwiązanie bez wskazywania agentowi endpointów ani modułów. Z odkrytej mapy wyprowadź konfigurację projektu i specyfikacje wszystkich rozpoznanych funkcji. Zakres obejmuje aplikację, testy i narzędzia, nie materiały szkoleniowe jako funkcje produktu. Nie zmienia ograniczeń dostępu zapisanych w instrukcjach projektu.

Brownfield oznacza pracę z istniejącym systemem, którego zachowanie trzeba najpierw poznać. W tym dodatku „migracja" tworzy opisy tego systemu, nie przenosi danych ani nie przebudowuje kodu. Spec Kit porządkuje specyfikację, plan i zadania; wygenerowany dokument nadal wymaga porównania z implementacją.

1. Zainstaluj Spec Kit przez pip Użyj **istniejącego `.venv` aplikacji**. Nie twórz drugiego środowiska ani nie instaluj `uv` lub `npm`. Otwórz PowerShell w głównym folderze aktualnej kopii projektu — tam, gdzie są `.venv`, `triage` i `starter`. Polecenia poniżej używają ścieżek względem tego folderu, więc nie trzeba aktywować środowiska ani ustawiać dodatkowych zmiennych.

Prowadzący wcześniej potwierdza dostępność `specify-cli` i zależności w skonfigurowanym firmowym rejestrze lub proxy. Korzystaj z tych ustawień.

Zainstaluj lub zaktualizuj Spec Kit:

```
.\.venv\Scripts\python.exe -m pip install --upgrade specify-cli
```

Sprawdź, czy polecenie działa:

```
.\.venv\Scripts\specify.exe version
```

Teraz przejdź do inicjalizacji w tym samym folderze.

2. Zainicjalizuj narzędzie we własnej kopii projektu W tym samym folderze uruchom:

```
.\.venv\Scripts\specify.exe init --here --integration copilot --script ps
```

`--here` oznacza bieżący folder, nie nowy projekt obok. Przed potwierdzeniem scalenia niepustego katalogu przejrzyj konflikty. Zachowaj własne instrukcje C1, skille i konfigurację. Nie dodawaj bezwarunkowo `--force`; jeżeli wybrana wersja go wymaga, najpierw zabezpiecz pliki i zatwierdź konkretny zakres nadpisania.

```
git status --short
git --no-pager diff
Get-ChildItem .\.specify
Get-ChildItem .\.github\skills -Directory
```

Obejrzyj też nowe, nieśledzone pliki — `git diff` ich nie pokazuje. Oczekuj `.specify` oraz skilli `speckit-*`. Sama inicjalizacja nie generuje specyfikacji z kodu ani nie implementuje funkcji.

3. Dodaj rozszerzenie Brownfield Brownfield Bootstrap jest rozszerzeniem społecznościowym, nie częścią utrzymywaną przez GitHub. Wersja `1.0.0` deklaruje Spec Kit >=0.4.0. Po zatwierdzeniu źródła rozszerzenia i dostępu sieciowego zainstaluj je w tym samym projekcie:

```
.\.venv\Scripts\specify.exe extension add brownfield `
  --from https://github.com/Quratulain-bilal/spec-kit-brownfield/archive/refs/tags/v1.0.0.zip
```

Istnieje kilka narzędzi odtwarzających specyfikację na podstawie kodu. W tym ćwiczeniu używamy jednego wskazanego rozszerzenia, aby zachować spójną procedurę; nie oznacza to, że jest jedynym rozwiązaniem.

Otwórz nową sesję Copilota w swojej kopii projektu. Poniższe wywołania wklejaj **do czatu agenta**, pojedynczo, nie do PowerShell. Nowy Spec Kit dla Copilota domyślnie udostępnia skille z myślnikami: `/speckit-brownfield-scan` i `/speckit-specify`. Dokumentacja rozszerzenia pokazuje starszą notację z kropkami, np. `/speckit.brownfield.scan`. Nie mieszaj trybów: sprawdź rzeczywiście wygenerowane pliki `.github\skills\speckit-*\SKILL.md` i nazwy dostępne w swojej sesji. Brak skilla oznacza problem instalacji lub wykrywania, nie wykonanie kroku.

4. Rozpoznaj całą aplikację i odtwórz specyfikacje Nie podawaj nazw funkcji, endpointów ani zakładanej architektury. Uruchom w głównym folderze kopii projektu sam skan:

```
/speckit-brownfield-scan
```

Skan jest tylko do odczytu. Jego wynikiem jest profil projektu w rozmowie, nie komplet specyfikacji. Przejrzyj wykryty stos, punkty wejścia, moduły, zależności, testy i narzędzia uruchamiania. Rozróżnij kod produktu, narzędzia pomocnicze i dokumentację. Nie zakładaj, że lista katalogów wyjaśnia już wszystkie zachowania aplikacji.

Zanim zatwierdzisz generowanie, uzupełnij rozpoznanie:

```
Na podstawie skanu rozpoznaj całe rozwiązanie, nie jeden wybrany przepływ.
Prześledź odkryte punkty wejścia, przepływy danych, zależności i testy.
Przedstaw mapę modułów oraz listę istniejących funkcji z dowodami
w kodzie i testach. Uwzględnij kontrakty, konfigurację i obsługę błędów.
Oddziel zachowanie wynikające z kodu, wyniki faktycznie wykonanych prób,
wymagania dokumentacji oraz braki i niewiadome.
Każdy obszar przypisz do funkcji albo narzędzi pomocniczych;
obszary nierozpoznane oznacz jawnie, zamiast pomijać je w podsumowaniu.
Nie zmieniaj plików. Nie projektuj ani nie implementuj nowych funkcji.
```

To doprecyzowanie oczekiwanego wyniku, nie lista odpowiedzi do odnalezienia. Jeżeli analiza jest niepełna, uzupełnij ją przed generowaniem. Dokumentacja wymagań służy do porównania z kodem, nie zastępuje dowodów implementacji. Obecność testu nie oznacza, że go uruchomiono.

Konfiguracja całego projektu:

Konstytucja to zbiór zasad, według których Spec Kit przygotowuje specyfikacje i plany. Ma odzwierciedlać uzgodnione standardy projektu, nie utrwalać każdego zastanego rozwiązania jako wzorca do naśladowania.

```
/speckit-brownfield-bootstrap
Wykorzystaj profil i mapę całego rozwiązania z poprzedniej analizy.
Uwzględnij starter\brief.md i starter\quality-standards.md
jako wymagania, a nie dowód istniejącego zachowania.
Przygotuj konstytucję i szablony zgodne z odkrytym projektem.
Zachowaj istniejące instrukcje. Pokaż plan scalenia
i poczekaj na zgodę przed zapisem.
```

Po zaakceptowanym scaleniu obejrzyj konstytucję, szablony i ewentualny `AGENTS.md`, a następnie sprawdź konfigurację:

```
/speckit-brownfield-validate
```

Rozbieżności konfiguracji z kodem wyjaśnij i popraw przed migracją, bez zmieniania aplikacji tak, aby pasowała do wygenerowanego opisu. Ta walidacja nie zastępuje testów aplikacji.

Specyfikacje odkrytych funkcji:

```
/speckit-brownfield-migrate all
```

W Brownfield `1.0.0` argument `all` oznacza odkrycie i przedstawienie listy funkcji do wyboru, **nie automatyczny zapis wszystkiego**. Porównaj listę z mapą całego rozwiązania. Wybierz wszystkie rozpoznane funkcje aplikacji i zatwierdzaj ich migrację kolejno. Nazwy oraz granice ustal z kodu, nie z przygotowanej wcześniej listy. Nie twórz osobnej funkcji biznesowej z każdego pliku pomocniczego.

Dla każdej funkcji oczekuj osobnego katalogu `specs\<nazwa-funkcji>` z `spec.md`, `plan.md` i `tasks.md`. Przed zapisem sprawdź plan plików i konflikty z istniejącymi artefaktami. Zachowaj `status: migrated`. Plan ma odtwarzać istniejącą implementację, a nie projektować jej przebudowę; luki nie są ukończonymi zadaniami nowej funkcji. Porównaj kontrakty i zależności między specyfikacjami. Jeśli znaleziono tylko jedną funkcję, nie dziel jej sztucznie na kilka.

Po migracji utrwal mapę, aby wynik nie pozostał tylko w rozmowie:

```
Przygotuj specs\existing-system-map.md na podstawie pełnej analizy
i wygenerowanych specyfikacji. Zapisz punkty wejścia, moduły,
zależności i przepływy oraz względne linki do specyfikacji funkcji.
Powiąż każdy rozpoznany obszar z kodem i testami; wskaż też narzędzia
pomocnicze, niewiadome i obszary bez specyfikacji wraz z przyczyną.
Oddziel stan istniejący, wymagania, braki i wyniki wykonanych sprawdzeń.
Nie uznawaj zadań [x] za dowód uruchomienia testów.
Pokaż plan zapisu lub scalenia i poczekaj na zgodę.
Nie zmieniaj kodu aplikacji.
```

Przejrzyj mapę i wszystkie wygenerowane artefakty. Pełny brownfield kończy się pokryciem rozpoznanej aplikacji, a nie jedną wybraną specyfikacją. Nieopisane obszary pozostają jawną luką rozpoznania. Nowe wymagania nadal wymagają osobnej specyfikacji w kolejnym zadaniu.

5. Skopiuj opis do katalogu specyfikacji i ustaw aktywną funkcję Użyj mapy `specs\existing-system-map.md`, aby znaleźć specyfikację przepływu potrzebnego do kolejnego zadania konsolowego. Zachowaj mapę i specyfikacje pozostałych funkcji; wybór aktywnej specyfikacji nie zawęża ani nie zastępuje analizy całej aplikacji. Jeśli kontekst nowej funkcji obejmuje kilka specyfikacji, powiąż je względnymi linkami i przeczytaj je przy jej opisywaniu.

Samo istnienie pliku nie ustawia kontekstu kolejnych poleceń Spec Kit. Zachowaj wynik migracji jako źródło, a sprawdzoną kopię umieść w `specs\001-triage-baseline`. Poniższe polecenie zapyta o rzeczywistą ścieżkę katalogu odkrytej specyfikacji i zapisze ją w `$source`. Jeśli katalog docelowy już istnieje, zatrzymaj się i wybierz wolny numer; nie nadpisuj wcześniejszej pracy.

```powershell
$source = Read-Host "Podaj sciezke katalogu specyfikacji wybranej z mapy"
$target = ".\specs\001-triage-baseline"
if (Test-Path $target) { throw "Katalog docelowy juz istnieje." }
foreach ($name in @("spec.md", "plan.md", "tasks.md")) {
    if (!(Test-Path (Join-Path $source $name))) {
        throw "Brak pliku migracji: $name"
    }
}
New-Item -ItemType Directory -Path $target -ErrorAction Stop
foreach ($name in @("spec.md", "plan.md", "tasks.md")) {
    Copy-Item (Join-Path $source $name) $target -ErrorAction Stop
}
```

Przejrzyj w kopii odwołania do starego katalogu i popraw je, jeśli mają wskazywać na pozostałe skopiowane artefakty. Nie usuwaj oznaczenia `status: migrated`. Następnie obejrzyj istniejący `.specify\feature.json`, jeśli jest obecny. Po świadomym zatwierdzeniu zmiany aktywnej funkcji zapisz poniższą zawartość tego pliku:

```json
{
  "feature_directory": "specs/001-triage-baseline"
}
```

Ścieżka powyżej odpowiada katalogowi `$target` z przykładu. Jeśli wybrałeś inny numer lub nazwę, wpisz rzeczywistą ścieżkę swojej kopii. `feature_directory` oznacza katalog aktywnej specyfikacji, nie nazwę funkcji Pythona. Do wyboru właściwej specyfikacji możesz użyć Copilota:

```
Przeczytaj specs\existing-system-map.md i #iss009.
Wskaż specyfikację istniejącej funkcji potrzebnej do zadania konsolowego
oraz katalog jej sprawdzonej kopii. Uzasadnij wybór na podstawie mapy.
Zaproponuj wartość feature_directory w .specify\feature.json.
Zapytaj mnie o zgodę przed zmianą pliku; po zgodzie ustaw tę wartość.
Nie nadpisuj specyfikacji ani nie implementuj zadania konsolowego.
```

Jeżeli ustawiałeś `SPECIFY_FEATURE_DIRECTORY`, usuń jego nadpisanie z terminala. Następnie uruchom z niego nową sesję agenta, aby również ona korzystała z pliku:

```powershell
Remove-Item Env:SPECIFY_FEATURE_DIRECTORY -ErrorAction SilentlyContinue
.\.specify\scripts\powershell\check-prerequisites.ps1 -Json -PathsOnly
```

Wynik ma wskazywać właściwy folder w `FEATURE_DIR` i plik w `FEATURE_SPEC`. Otwórz ten plik i sprawdź jego treść. Poniższy zrzut ilustruje format odpowiedzi, nie ścieżkę do skopiowania:



Rysunek 3: Przykładowy wynik kontroli ścieżek aktywnej specyfikacji Spec Kit.

Zadanie końcowe — TUI: komentarz z konsoli i ocena JSON

Opcjonalne zadanie: `#iss009`. Sprawdź zachowanie według AC-11. Zacznij po etapach 1–8 i przygotowaniu specyfikacji stanu istniejącego z `#iss010`.

Nowa funkcja: użytkownik uruchamia aplikację w konsoli, wpisuje `store_id` i tekst komentarza, a aplikacja wypisuje ocenę jako JSON — taki sam kontrakt jak odpowiedź dla formularza WWW. To prosty interaktywny interfejs tekstowy, bez pełnoekranowego frameworka, bez nowych pakietów i bez monitora postępu ewaluacji.

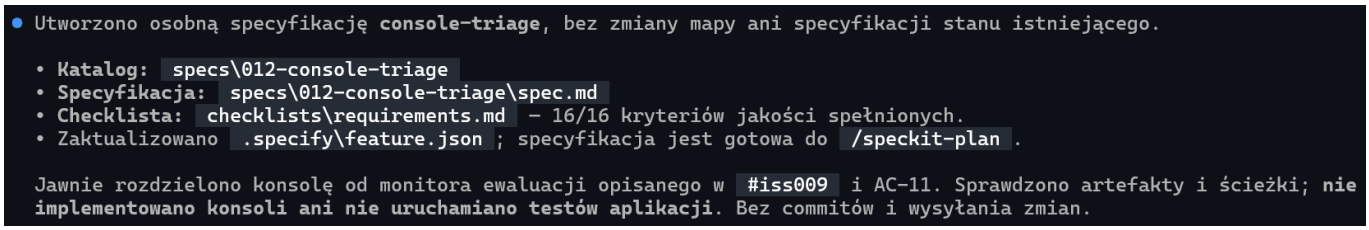
Docelowe polecenie to `python -m triage.console`. Moduł dopiero utworzysz; nie jest gotowym poleceniem startera. W jednym uruchomieniu wprowadzasz jeden komentarz. Pytania i komunikaty trafiają do `stderr`, a na `stdout` po sukcesie trafia jeden poprawny obiekt JSON. Proces kończy się kodem `0`; kod wyjścia nie jest częścią JSON. `stdout` to standardowe wyjście programu, a `stderr` — osobny kanał komunikatów. Ten podział pozwala zapisać sam wynik do pliku bez pytań o dane wejściowe. Nie jest to eksport samego `jira_draft`: wynik obejmuje również `no_action` i `manual_review`, wtedy ze szkicem `null`. Przy błędzie nie wypisuj wyniku sukcesu ani starej oceny: czytelny komunikat na `stderr`, pusty `stdout`, kod `1`. EOF lub przerwanie przed wynikiem kończy pracę bez klasyfikacji niedokończonego wejścia, bez JSON i z kodem `2`.

1. Zapisz osobną specyfikację nowej funkcji

```
/speckit-specify
Odczytaj #iss009.
Odczytaj również specs\existing-system-map.md
i powiązane specyfikacje potrzebne do tej zmiany.
Dodaj nowy interfejs konsolowy: python -m triage.console.
Użytkownik wpisuje store_id i komentarz, a otrzymuje JSON oceny
zgodny z aktualnym POST /api/triage, jak dla formularza WWW.
Uwzględnij AC-11 i opcjonalne wymaganie w starter\brief.md.
Utwórz osobną specyfikację nowej funkcji console-triage,
nie nadpisuj opisu stanu istniejącego.
```

Sprawdź nowy katalog `specs` i `.specify\feature.json`: aktywna funkcja ma wskazywać **nową** specyfikację, nie `001-triage-baseline`. Numer nowego katalogu wybiera narzędzie; używaj rzeczywiście zwróconej ścieżki. Uruchom ponownie `check-prerequisites.ps1 -Json -PathsOnly` i otwórz wskazany `FEATURE_SPEC`.

Przykładowy wynik utworzenia nowej specyfikacji:



Rysunek 4: Przykładowe podsumowanie utworzenia osobnej specyfikacji interfejsu konsolowego.

2. Doprecyzuj, zaplanuj i dopiero wtedy implementuj Wszystkie poniższe polecenia wpisuj do czatu **pojedynczo**. Po każdym przeczytaj wynik, zanim przejdziesz dalej. Jeśli specyfikacja pozostawia niejasności, skorzystaj z opcjonalnego kroku:

```
/speckit-clarify
```

Następnie utwórz plan:

```
/speckit-plan
```

Po jego przeglądzie utwórz zadania:

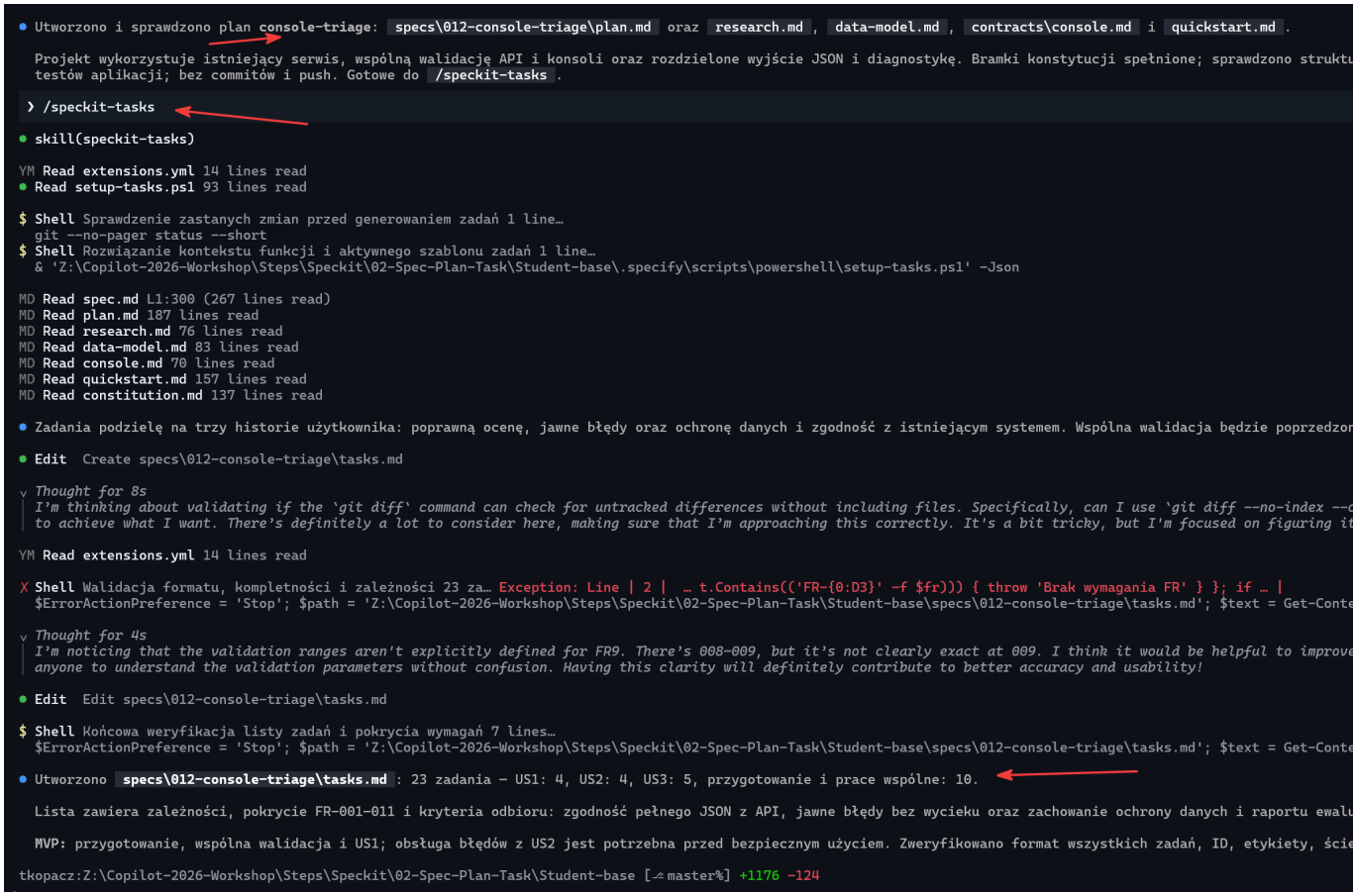
```
/speckit-tasks
```

Plan i zadania są wymagane w tym opcjonalnym rozszerzeniu. Nie oznacza to włączenia Spec Kit ani TUI do obowiązkowego rdzenia.

Opcjonalnie sprawdź spójność artefaktów:

```
/speckit-analyze
```

Niezależnie od użycia `clarify` sprawdź sposób kończenia wpisu: w tym ćwiczeniu jeden wiersz na `store_id` i jeden na komentarz. Jeśli potrzebujesz tekstu wielowierszowego, najpierw uzgodnij zmianę specyfikacji. Plan ma wykorzystywać



Rysunek 5: Przykładowe podsumowanie planu i zadań dla interfejsu konsolowego.

tę samą usługę i walidację co WWW, bez kopiowania reguł do konsoli i bez obowiązku uruchamiania serwera HTTP. Zachowaj tryby `demo`/`live`, błędy konfiguracji i brak automatycznego przełączania na `demo`. Kontrakt WWW i raport ewaluacji nie zmieniają się.

Sprawdź, czy zadania obejmują test porównujący JSON konsoli i API dla identycznego wejścia oraz kontrolowanej odpowiedzi klasyfikatora. To sprawdzenie wynika z AC-11; opcjonalność polecenia `analyze` nie czyni kryteriów jakości opcjonalnymi. Przed implementacją test ma wykazać brak wymaganego zachowania, nie tylko awarię importu. Usuń istotne sprzeczności znalezione podczas przeglądu lub przez `analyze`, a następnie zatwierdź plan i zleć implementację:

`/speckit-implement`

Przykładowe podsumowanie implementacji, jeszcze przed dodatkowym sprawdzeniem:

Po przeczytaniu wyniku uruchom drugi wymagany krok tej ścieżki:

`/speckit-converge`

Możesz użyć mniejszego modelu; w przykładowej sesji wybór Auto uruchomił Lunę. Nie zakładaj, że Auto zawsze wybierze ten sam model. Tak jak przy krytyce C5, sprawdź uwagi na podstawie kodu i wymagań.

Jeżeli `converge` wskazuje braki, popraw zadania, ponów implementację i sprawdzenie. Sam plan, `[x]` lub werdykt narzędzia nie zastępuje uruchomienia konsoli.

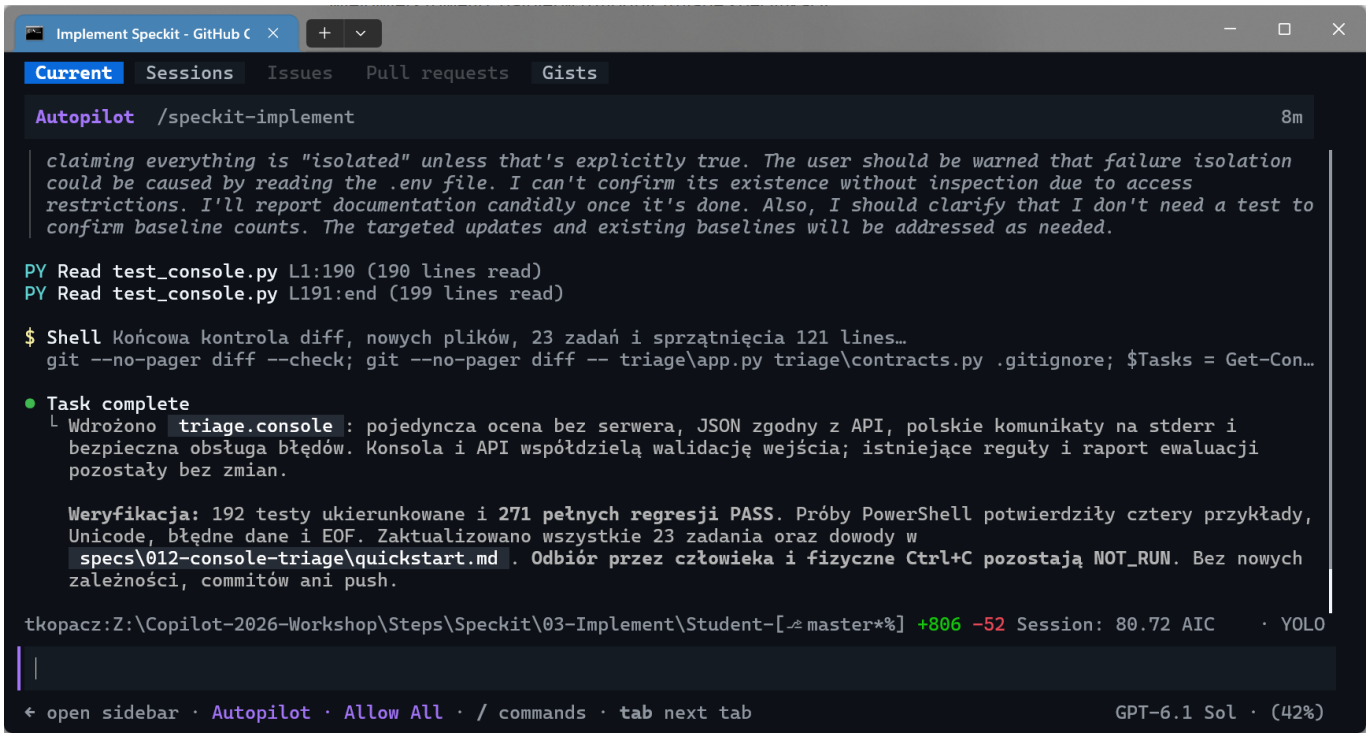
3. Samodzielnie wypróbuj konsolę Po implementacji uruchom w terminalu ze skonfigurowanym trybem `demo`:

```
.\.venv\Scripts\python.exe -m triage.console
```

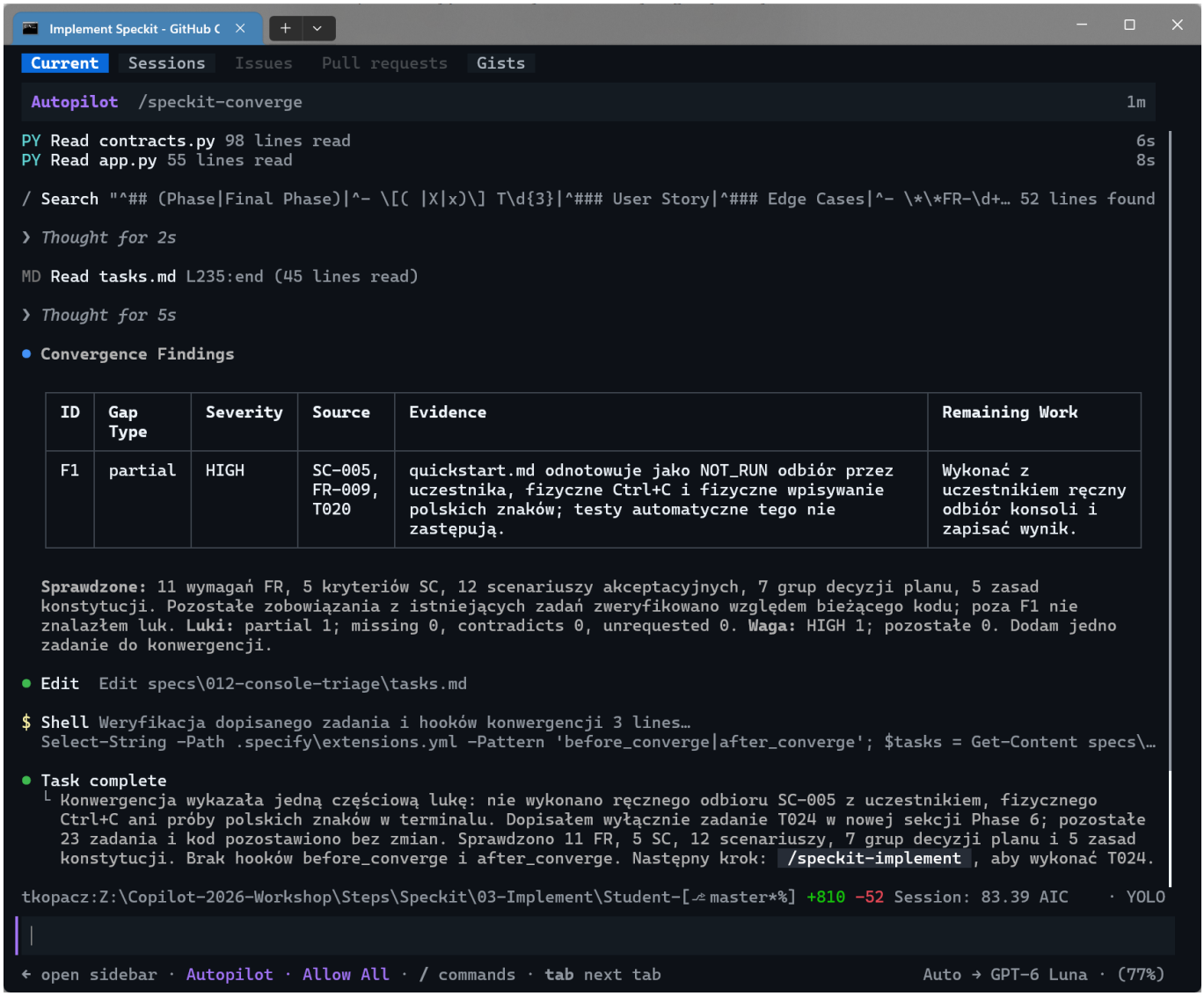
Wpisz `store-001`, a następnie „Miła obsługa, ale składanie zamówień nie działa.”. Oczekuj JSON z `order`, `P1`, `actionable`, `draft_ready` i szkicem, bez wysłania zgłoszenia. Powtórz dla pochwały i niejasnej sprawy. Sprawdź także przekierowanie samego wyniku. Użyj nowej nazwy pliku: operator `>` nadpisuje istniejącą zawartość.

```
.\.venv\Scripts\python.exe -m triage.console > console-result.json
$LASTEXITCODE
Get-Content .\console-result.json -Raw | ConvertFrom-Json
```

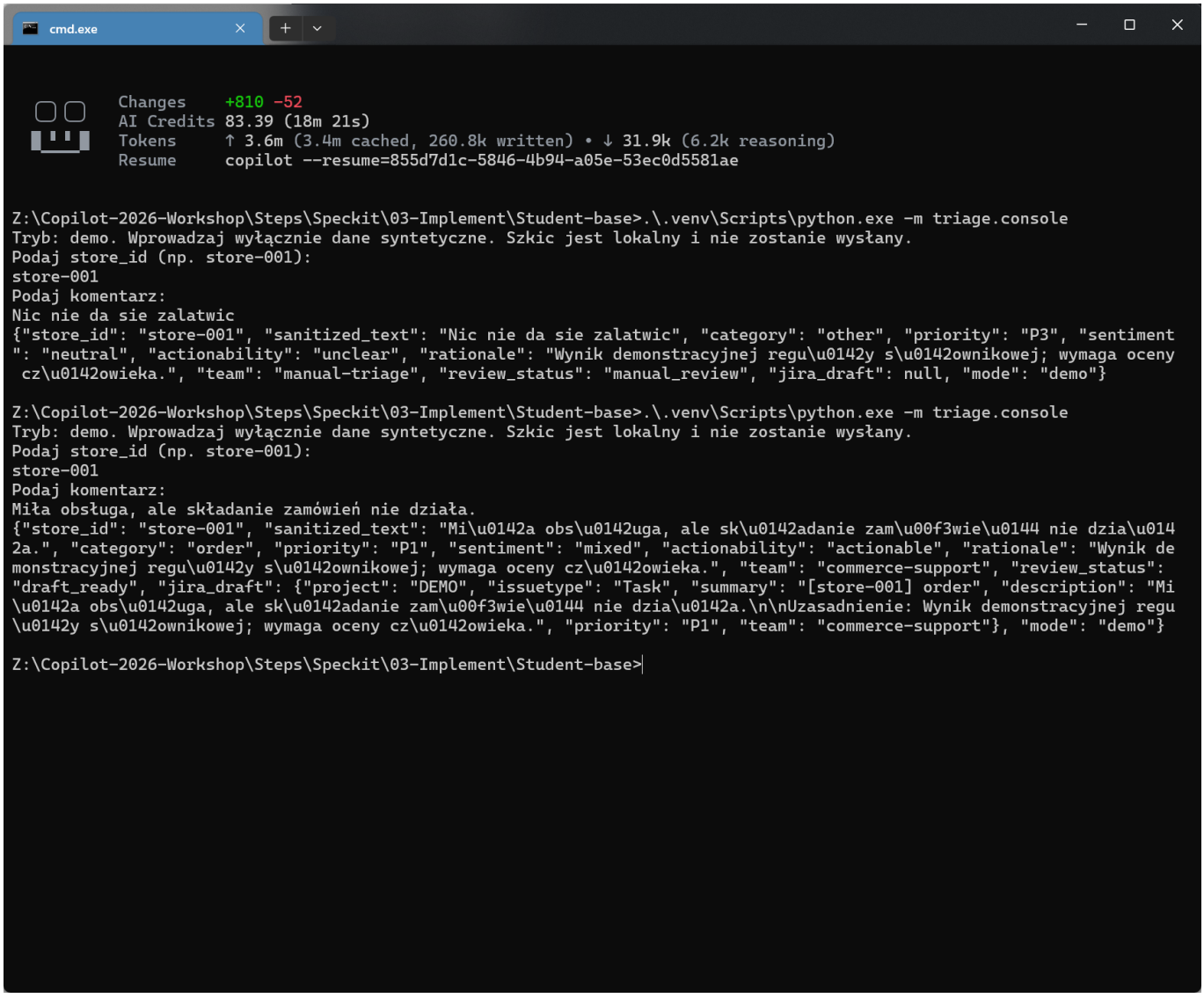
Przykładowy przebieg próby konsoli:



Rysunek 6: Przykładowe podsumowanie implementacji interfejsu konsolowego przed krokiem converge.



Rysunek 7: Przykładowy wynik kroku converge z mniejszym modelem.



Rysunek 8: Przykładowa ocena komentarza i kontrola wyniku interfejsu konsolowego.

Pytania nadal powinny być widoczne, a plik ma zawierać tylko JSON. Na kontrolowanym klasyfikatorze porównaj wszystkie pola i wartości JSON z API; w `live` nie wymagaj identycznego swobodnego uzasadnienia z dwóch niezależnych wywołań modelu. Osobno sprawdź pusty tekst, tekst >4000 znaków, niepoprawny sklep, awarię dostawcy, EOF/przerwanie i wymagane maskowanie. Kod `0` dotyczy także poprawnej oceny `manual_review` lub `no_action`; nie oznacza wysłania szkicu. Przy błędzie oczekuj kodu `1`, a przy przerwaniu `2`, bez JSON sukcesu. Do tego dodatku możesz wrócić później; nie zmienia to tego, co już zrobiłeś i sprawdziłeś w głównych etapach. EOF oznacza koniec wejścia, a nie pusty komentarz. W konsoli Windows zwykle wyślesz go przez `Ctrl+Z`, a następnie `Enter`; `Ctrl+C` służy przerwaniu. Sprawdź oba przypadki osobno, także przed wprowadzeniem drugiego pola.

Źródła poleceń: instalacja Spec Kit, integracje i notacja skilli, Brownfield Bootstrap.

Co wykorzystasz przy następnej pracy z AI

Przy kolejnym zadaniu wróć do tych nawyków:

1. Przed zmianą ustal, czego oczekujesz i jak to sprawdzisz.
2. Zobacz, jakie instrukcje i źródła agent rzeczywiście wykorzystał.
3. W nowej sesji sprawdź notatkę z bieżącym kodem, zamiast ufać jej w ciemno.
4. Przeczytaj plan i różnice w plikach przed przyjęciem zmian.
5. Uruchom testy i sam wypróbuj działanie aplikacji, również po błędzie.
6. Porównuj pomiary wykonane w tej samej konfiguracji.
7. Sprawdź uwagi drugiego agenta — on też może się mylić.
8. Porównaj podsumowanie automatyzacji z jej rzeczywistymi wynikami.

AI pomaga wykonać pracę. Ty sprawdzasz, co powstało, czy działa i czy odpowiada twojemu zamiarowi.

Przydatne polecenia

```
.\scripts\test.ps1 -PytestArgs @("tests\test_privacy.py", "-q")
.\scripts\test.ps1
.\scripts\run.ps1 -Port 5050
.\scripts\smoke.ps1 -Port 5050
.\.venv\Scripts\python.exe -m pytest
.\.venv\Scripts\python.exe -m triage.evaluate
```